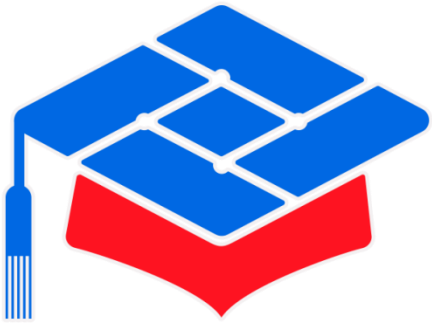


第十届

全国大学生集成电路创新创业大赛

报告类型：技术文档

参赛杯赛：“紫光同创”企业命题

作品名称：基于紫光 FPGA+RK3568 的视频流智能交通识别系统

队伍编号：CICC1001862

团队名称：ZLC

目录

1. 作品概述.....3
2. 系统组成及总体设计方案.....5
   2.1 整体介绍.....5
   2.2 硬件系统介绍.....6
      2.2.1 硬件整体介绍.....6
      2.2.2 电路各模块介绍.....6
   2.3 算法介绍.....9
      2.3.1 算法整体介绍.....9
      2.3.2 RK3568 算法模块介绍.....9
      2.3.3 FPGA 端算法模块介绍.....26
3. 性能参数.....35
   3.1 FPGA 端车牌检测系统误差分析与资源消耗.....35
   3.2 RK3568 侧误差测算.....38
      3.2.1 模型训练概述.....38
      3.2.2 实验配置.....39
      3.2.3 模型的量化与部署.....40
4. 仿真与测试报告.....41
   4.1 系统模块功能仿真测试.....41
   4.2 仿真环境与参数设置.....41
   4.3 模块仿真结果分析.....42
5. 总结.....47
   5.1 可扩展之处.....47
   5.2 心得体会.....47
6. 主要代码解析.....48
7. 参考文献.....51
8. AI 使用说明.....52

1. 作品预览简介

本项目设计了一套基于FPGA与ARM异构架构的智能交通视频流分析系统。系统采用盘古 PG2L200H-6IFBG676 与 RK3568 作为核心硬件平台，主要实现两大核心功能：基于视频流的实时车牌检测与识别、基于视频流的行人闯红灯检测。通过模块化设计思路，系统将计算任务合理分配：FPGA 负责前端高速图像预处理与目标初步框选定位，RK3568 负责后端高精度深度学习推理与结果渲染输出，从而构建出高效的并行处理视频分析架构。

关键技术：

- **FPGA 硬件加速与流水线处理：**构建了完整的底层图像预处理流水线，涵盖色彩空间转换、双重颜色掩码二值化、3x3 中值滤波、形态学处理及垂直 Sobel 边缘提取。同时开发了底层的 UDP 硬件协议栈、RGMII 与 GMII 接口转换，以及基于 AXI 总线的 DDR3 跨时钟域显存缓冲架构，保障了高吞吐量视频流的稳定传输。
- **双目标动态追踪与 BRAM 帧差计算：**在 FPGA 端独立设计了 super_plate_tracker 追踪模块，利用像素级连通域合并和四角极值算法实现车牌目标的容错提取。针对行人检测，采用带有读写地址指针的 4-bitBRAM 进行帧差计算，确保前后两帧像素在物理坐标系上的绝对严格对齐。
- **RK3568 边缘 AI 部署与推理：**RK3568 端部署改进的 YOLOv5s 检测模型与 CTC 端到端文字识别模型。采用多进程与多线程并发架构，利用队列分离 I/O 密集型的视频采集子进程与计算密集型的主推理线程。NMS 等后处理外置于模型独立运行，避免了动态算子在 INT8 量化过程中带来的精度损失。
- **全栈远程管理系统：**采用前后端分离架构搭建 Web 远程管理平台。通过 SSH 和 WebSocket 协议，不仅支持对嵌入式端多种推理模式的远程启停控制，还能实现实时终端日志的低延迟推送与监控。

重要指标：

- **系统吞吐与接口能力：**支持接 640x480 的 UDP 以太网视频流或 1280x720@30fps 的 HDMI 采集卡 MJPG 视频输入。
- **模型量化精度：**算法模型经 Rknn-Toolkit2 进行 INT8 量化后，检测模型 mAP@0.5 达 0.935；识别模型整牌准确率达 0.936。
- **推理加速性能：**部署至端侧后，检测推理单帧耗时降至 50ms，识别推理单帧耗时降至 10ms，充分满足智能交通场景的实时性要求。
- **系统误差量化分析：**经对照实验测算，FPGA 端车牌追踪算法的水平方向系统误差约 28px，均值角点误差约 13.4px，垂直误差约 4px；误差主要源于形态学外扩操作，且算法展现出对 0-20 度轻度倾斜角的强鲁棒性。

作品亮点

- **创新的“串口辅助”软硬异构协同：**系统提供三种灵活的运行模式。在核心的“串口坐标快速识别”模式中，FPGA 在硬件流水线完成车牌物理框选后，将四角坐标通过 UART 串口直传至 RK3568。RK 端接收坐标后直接裁剪目标 ROI 区域并送入 CTC 识别模型，跳过高算力消耗的 YOLO 检测环节，释放了 NPU 算力并提升了系统响应速率。
- **鲁棒的特化硬件算法与容错机制：**针对车牌长条形物理特征，FPGA 底层定制了搜索半径长达 12 的一维水平膨胀算子，强行桥接断裂字符。在追踪机制中引入了 40 像素断裂缝隙容错、密度过滤，以及针对特殊车牌的自适应坐标向上矫正逻辑，全面适配复杂路况。
- **多源融合的高效抗干扰逻辑：**行人闯红灯预警模块深度集成了抗环境干扰策略。在帧差逻辑外围叠加了全局切镜防抖机制：当单帧内检测到运动像素超 20000 个时，系统立即判定为摄像头晃动或环境剧变并强制拉高异常标志位，有效避免了因物理震动产生的大规模误报。

2. 系统组成及总体设计方案

2.1 整体介绍

系统采用 FPGA+Arm 异构思路，采用盘古 PG2L200H-6IFBG676+RK3568 的硬件平台，核心实现两大功能：**车牌识别**和**行人闯红灯检测**。基于模块化设计思想，由 FPGA 负责高速图像预处理与实时车牌检测及行人闯红灯检测；RK3568 可支持完整的车牌检测与识别；同时可通过串口传输 FPGA 已定位的车牌坐标实现 RK 车牌识别。通过并行处理架构实现高效的视频流分析。核心部分包括：FPGA 端图像采集与预处理模块及车牌框选、RK3568 车牌检测与识别部分、行人闯红灯两种方法实现、DDR 内存模块、输出显示模块、以太网和 USB 的视频输出接口。

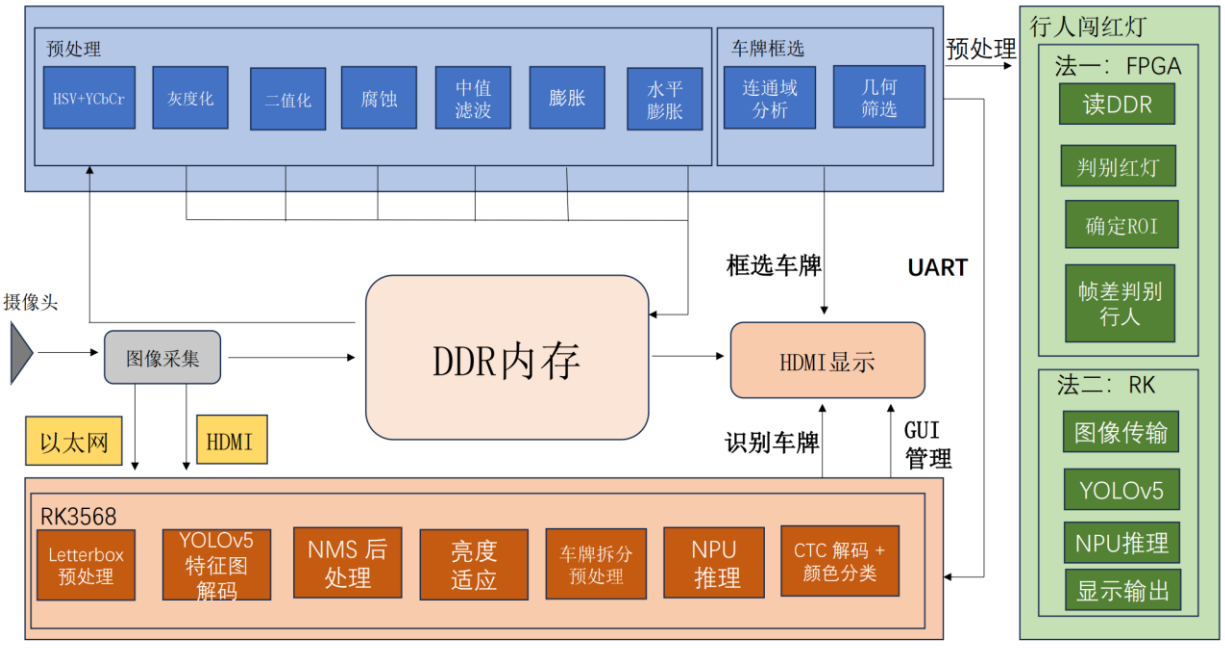


图 1 整体架构流程图

2.2 硬件系统介绍

2.2.1 硬件整体介绍

系统使用盘古 PG2L200H-6IFBG676[1]开发板，摄像头 OV5640[4]通过 FMC[3]扩展板与 FPGA 相连，摄像头采集车牌照片、行人闯红灯视频支持 DDR 读取在 FPGA 硬件[2]流水线处理完成车牌框选、行人闯红灯框定并输出 HDMI 显示；或经过以太网、HDMI-USB 转接口传入 RK3568 通过 NPU 单独完成高精度字符识别与行人检测；同时支持通过串口从 FPGA 向 RK 传输实时车牌坐标，帮助 RK 舍弃定位算法加速 RK 识别。

硬件接口示意如下：

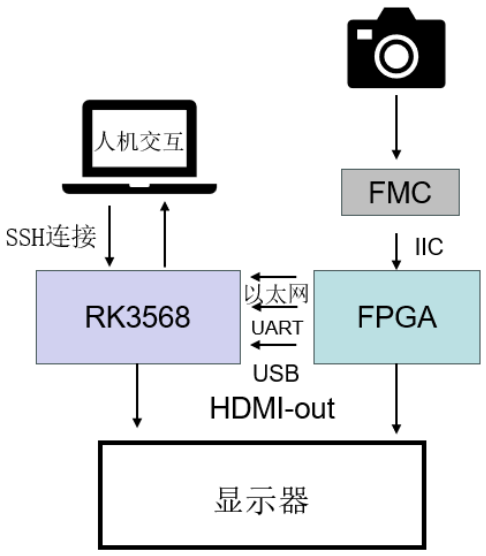


图 2 硬件接口示意图

2.2.2 电路各模块介绍

1.UART 电路：

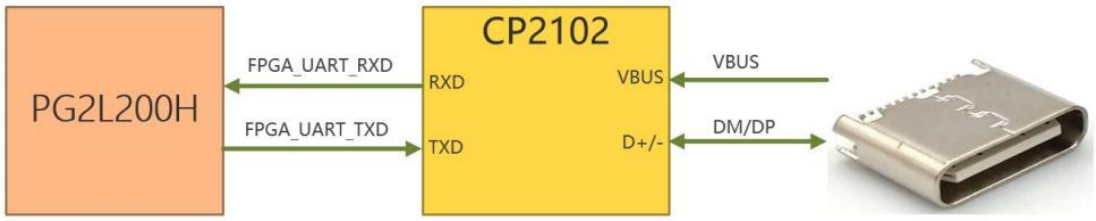


图 3 UART 电路

2.HDMI 电路：

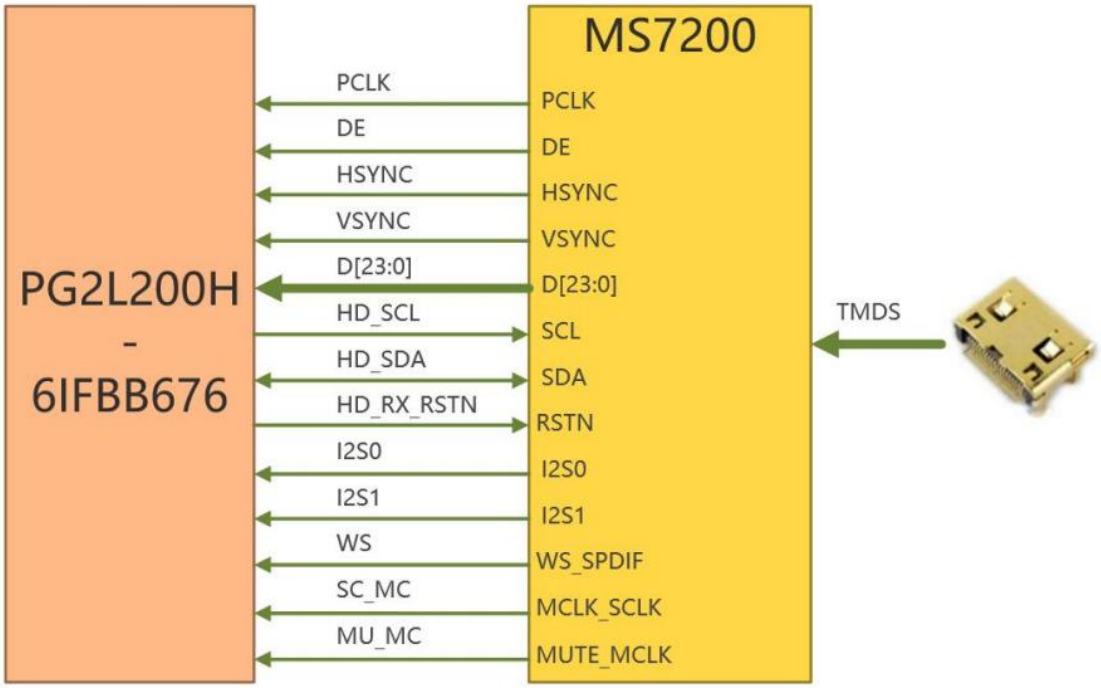


图 4 HDMI 电路图

3.FMC 连接器电路：

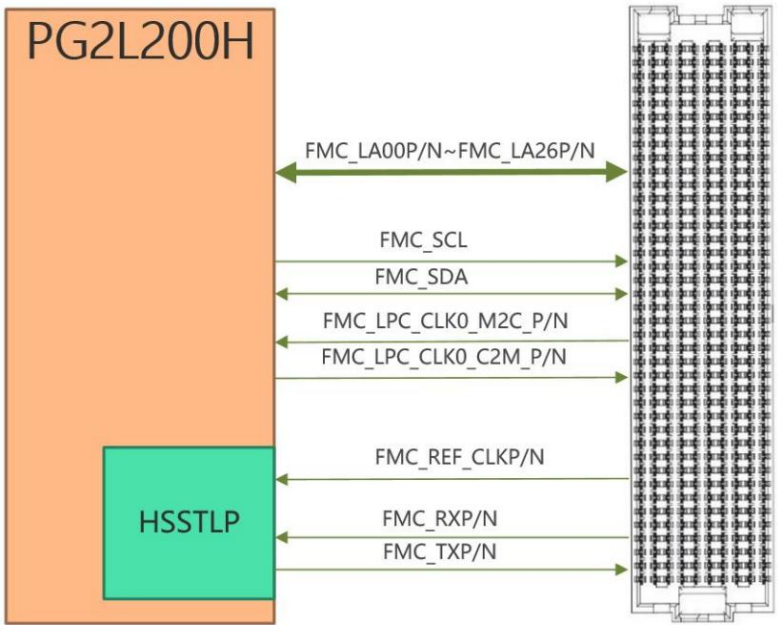


图 5 FMC 连接器电路

4.OV5640 摄像头电路：

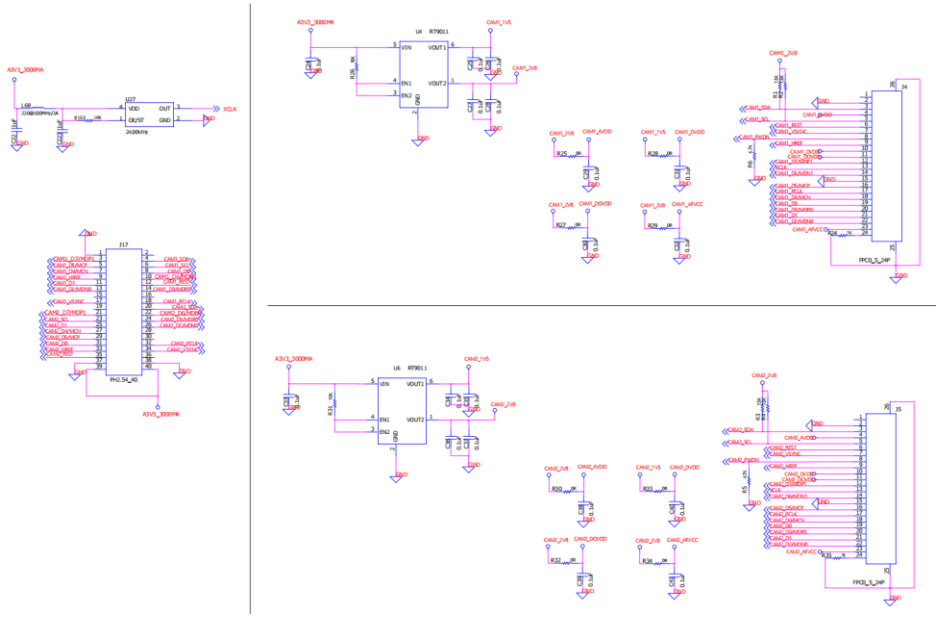
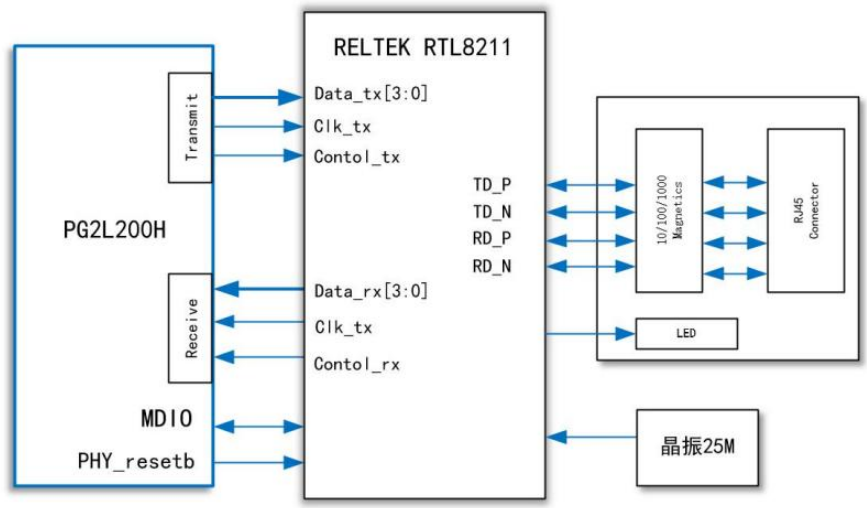


图 6 OV5640 电路图

5.网口连接框图：



网口连接框图

图 7 网口连接框

6.HDMI 转 USB 转接模块电路图：

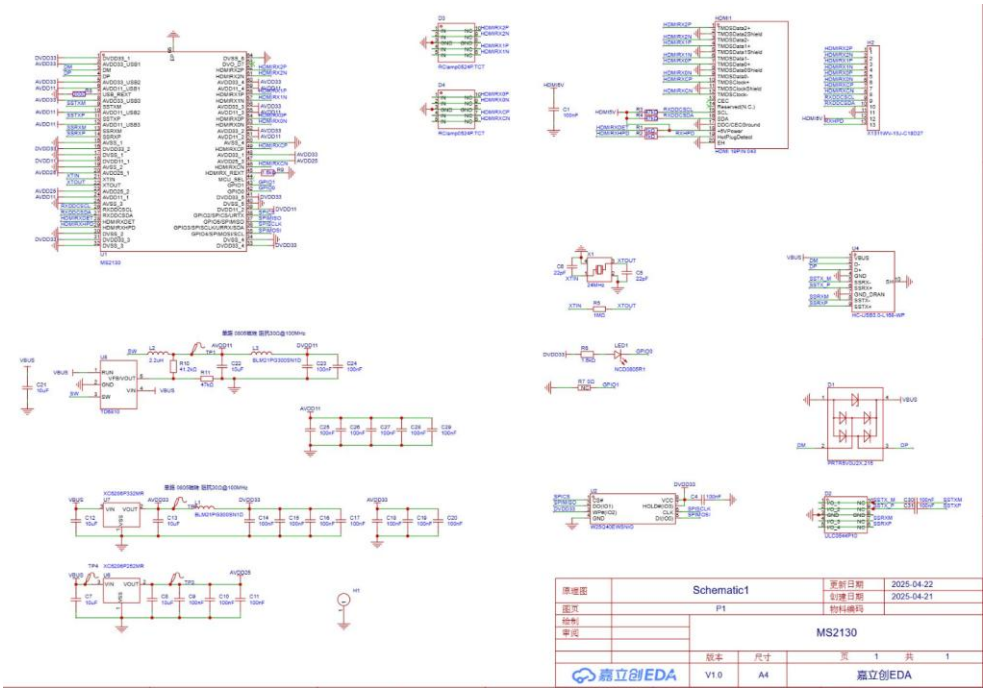


图 8 HDMI 转 USB 电路图

2.3 算法介绍

2.3.1 算法整体介绍

FPGA 端：负责视频采集，通过色彩空间转换、图像滤波、形态学处理、连通域分析、车牌角点检测等完成车牌框选；通过图像预处理、颜色判别、帧差检测完成行人闯红灯的违法识别。

RK3568 端：运行 YOLOv5、CTC、RKNN 等深度学习推理算法，完成车牌检测与识别、行人闯红灯行为分析。

2.3.2 RK3568 算法模块介绍

系统概述：

面向智能交通场景，实现两大核心功能：

- 基于视频流的实时车牌检测与识别。
- 基于视频流的行人闯红灯检测。

硬件平台由开发板、板卡、采集卡和显示器组成。系统代码分为部署在 RK3568 的嵌入式应用和部署在上位机的远程管理平台。

术语	说明
RKNN	Rockchip NPU 推理引擎及开发工具包
FPGA	现场可编程门阵列
NPU	神经网络处理单元
RGB565	16 位色深像素格式
CTC	连接时序分类算法
NMS	非极大值抑制
V4L2	Linux 视频采集框架
Framebuffer	Linux 内核帧缓冲设备
JWT	鉴权令牌
SSH	安全外壳协议

表 1 术语及说明

设计功能：

车牌检测与识别：通过以太网 UDP 或 HDMI 采集卡接收 FPGA 视频流，使用 RKNN NPU 运行 YOLOv5 检测模型和 CTC 识别模型，完成车牌定位、文字识别和颜色分类，结果通过 HDMI 输出。

串口坐标快速识别：接收 FPGA 通过串口发送的车牌四角坐标，跳过检测模型直接进行识别，提升处理速度。

行人闯红灯检测：使用 YOLOv5 模型检测行人、红绿灯及闯红灯行为，支持 FPGA 以太网、HDMI 采集卡和本地视频三种输入源。

Web 远程管理：提供基于浏览器的管理平台，通过 SSH 远程启动，停止嵌入式端功能，实时查看运行日志。

系统组成：

嵌入式 AI 应用：部署于 RK3568 开发板，负责视频采集、AI 推理和结果输出。

Web 远程管理平台：部署于上位机，提供 Web 界面进行远程控制和监控。

环境分类	组建/类别	规格与说明
硬件环境	主控板	RK3568
	FPGA 板卡	PG2L200H-6IFBG676
	HDMI 采集卡	MACROSILICON 芯片
	HDMI 显示器	通过/dev/fb0 Framebuffer 输出
软件环境	操作系统	Linux
	编程语言	Python3
	算法与视觉处理库	RKNN-Toolkit2、OpenCV、Pillow、NumPy
	硬件通信库	Pyserial
	前后端框架	Flask、Vue3、Element Plus

表 2 运行环境说明

总体架构：

系统采用三层架构，自下而上分别为感知层、处理层和应用层。

感知层：由摄像头、FPGA 板卡和 HDMI 采集卡组成。摄像头采集视频信号并传输给 FPGA 板卡，FPGA 负责视频编码与转发，通过以太网 UDP 协议将 RGB565 格式的 640×480 视频帧持续发送至 RK3568 开发板，同时通过 UART 串口发送车牌四角坐标数据。HDMI 采集卡作为备选输入源，通过 V4L2 接口提供 1280×720@30fps 的 MJPG 视频流。

处理层：部署在 RK3568 开发板上。该层运行嵌入式 AI 应用，采用多进程+多线程并发架构：独立子进程负责视频采集，通过单槽 multiprocessing.Queue 与主进程通信；主线程执行视频解码、NPU 推理和结果输出的完整流水线。RK3568 内置的 NPU 运行 RKNN int8 量化模型，完成 YOLOv5 目标检测和 CTC 文字识别等 AI 推理任务。处理结果通过 HDMI Framebuffer 输出至显示器。

应用层：包含 HDMI 视频输出终端和 Web 远程管理平台。Web 平台采用前后端分离架构，后端基于 Flask 框架，通过 Paramiko SSH 协议远程连接 RK3568 设备，控制推理模式的启停；前端为 Vue3 单页应用，通过 WebSocket 实时接收运行日志和状态推送。用户通过浏览器即可完成远程监控和管理。

各层之间通过以下协议通信：FPGA 与 RK3568 之间使用 UDP 以太网[]或 HDMI 传输视频帧和 UART 串口传输坐标数据；Web 管理平台与 RK3568 之间使用 SSH 协议进行命令控制，WebSocket 协议进行实时日志推送，HTTP REST API 进行模式管理和状态查询。

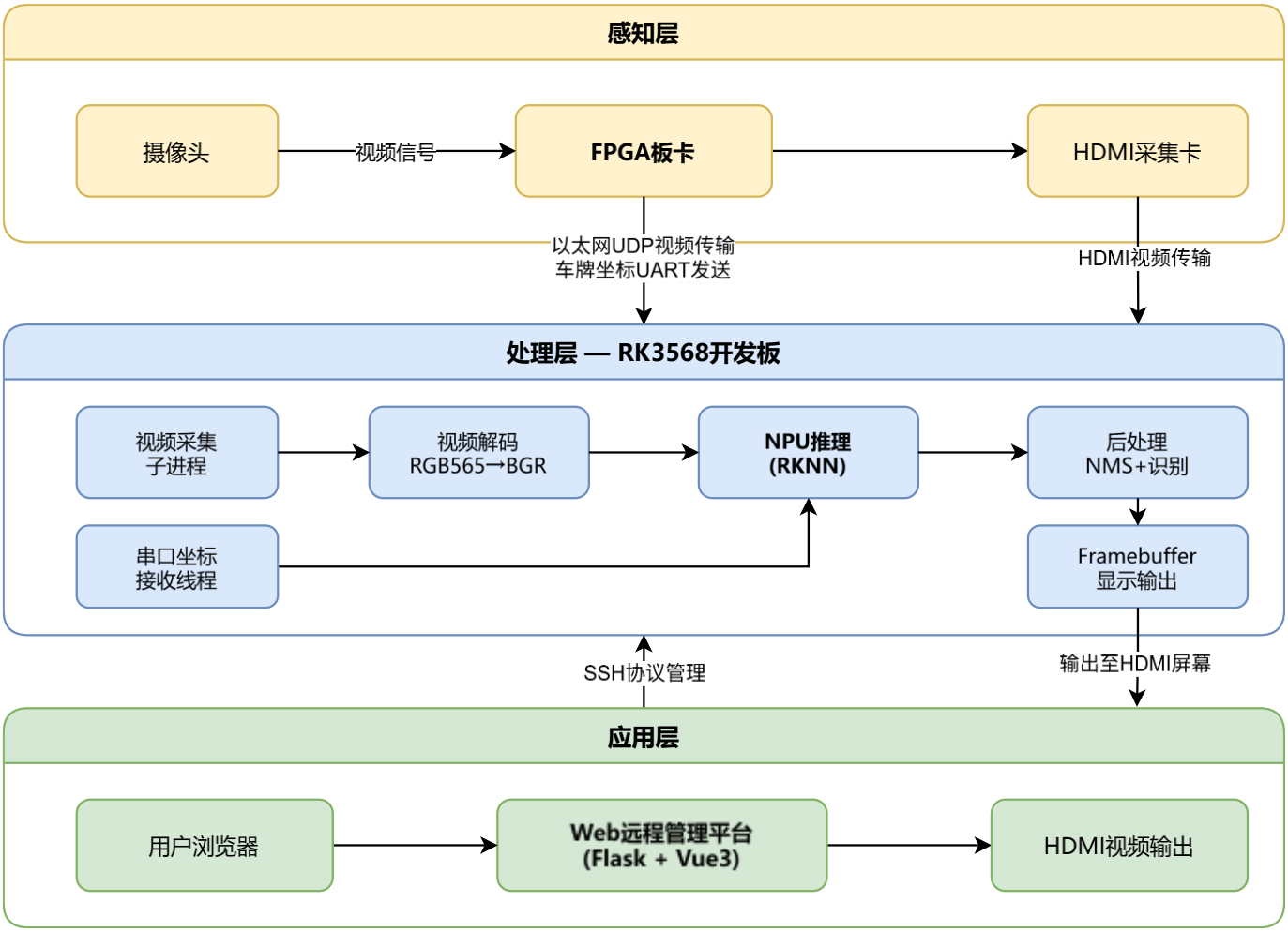


图 9 系统总体架构图

RK3568 架构

软件框架

final/	
├── core/	核心通信与基础模块
│ ├── fpga_io.py	FPGAUDP 视频流接收
│ ├── hdmi_io.py	HDMI 采集卡视频流接收
│ ├── serial_io.py	FPGA 串口坐标接收
│ ├── display.py	HDMIFramebuffer 显示输出
│ ├── postprocess.py	YOLO 后处理
│ └── image_utils.py	图像预处理工具
├── plate_det/	车牌检测与识别模块
│ ├── plate_detector.py	车牌检测推理
│ ├── plate_recognizer.py	车牌识别
│ ├── plate_pipeline.py	端到端处理流水线
│ ├── plate_draw.py	可视化绘制
│ └── plate_recognition.py	字典表与双层车牌处理
├── people_det/	行人闯红灯检测模块
│ ├── people_detector.py	RKNN 检测器类
│ └── people_run.py	多模式运行入口
├── udp_fpga_ai.py	全流水线入口
├── udp_fpga_serial.py	串口坐标模式入口
├── people_redlight.py	行人闯红灯检测入口
└── rknn_infer.py	CLI 批量离线推理工具

表 3 目录结构

嵌入式端 AI 应用部署在 RK3568 开发板的/home/cat/RK3568/final 目录下，采用 Python 语言开发，基于 RKNN-Toolkit 进行 NPU 推理。整个应用按功能划分为三层：应用入口层、AI 检测/识别层和核心基础层。

应用入口层：提供四个独立的运行入口，每个入口对应一种使用场景。

udp_fpga_ai.py：　　执行完整的车牌检测+识别流水线，接收 FPGA 以太网或 HDMI 采集卡视频流，依次调用检测模型和识别模型，结果输出至 HDMI；

udp_fpga_serial.py：　　跳过检测环节，接收 FPGA 串口发送的车牌坐标后直接调用识别模型，适用于 FPGA 已完成目标检测的场景；

people_redlight.py：　　运行行人闯红灯检测，支持 FPGA 以太网、HDMI 采集卡和本地视频文件三种输入源；

rknn_infer.py：　　CLI 批量离线推理工具，遍历图片目录执行检测+识别并将结果保存用于前期调试。

AI 检测/识别层：封装核心推理逻辑。

plate_detector.py：　　运行 RKNN 车牌检测模型检测车牌

plate_pipeline.py：　　车牌透视校正、亮度增强、双层车牌分割预处理

plate_recognizer.py：　　车牌识别 CTC 文字解码、颜色分类

plate_draw.py：　　绘制识别结果和中文标签

people_detector.py：　　运行 RKNN 检测模型检测行人、红绿灯，斑马线等目标

people_run.py：　　提供行人闯红灯检测的多模式运行封装

核心基础层：提供硬件通信和通用处理能力，被上层所有模块共同依赖。

fpga_io.py：　　实现 UDP 视频帧接收与 RGB565 解码；

hdmi_io：　　通过 V4L2 采集 HDMI 视频流；

serial_io.py：　　解析 UART 串口坐标协议；

display.py：　　通过 mmap 直接写入 Framebuffer 实现 HDMI 输出；

postprocess.py：　　提供 NumPy 实现的 YOLO 特征图解码和 NMS 后处理。

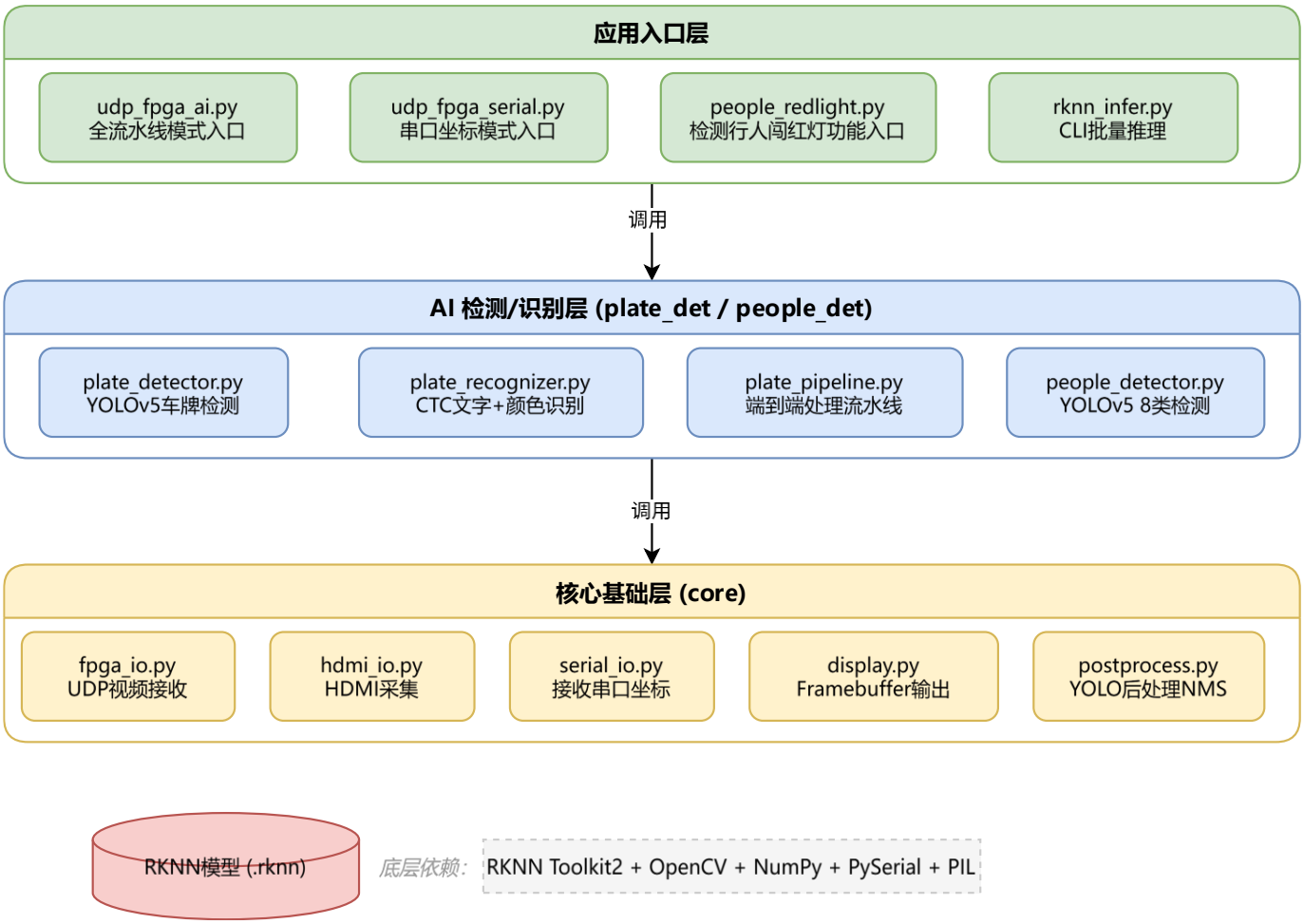


图 10 嵌入式端流程图

进程与线程模型：系统采用多进程+多线程的并发架构，针对不同的 I/O 密集型和计算密集型任务分别选用合适的并发机制。

进程 1：视频采集独立子进程

该进程负责从硬件设备持续采集视频帧并通过单槽队列传递给主进程。

UDP 模式下，进程维护一个 bytearray 缓冲区处理网络粘包问题，通过搜索帧头标记定位帧边界，截取完整的 614408 字节帧数据；

HDMI 模式下，进程通过 V4L2 接口以 MJPG 编码从采集卡读取 1280×720 帧。当队列已满时采用丢弃旧帧策略确保主进程始终获取最新画面。

Process 1:udp_receiver、hdmi_receiver

Input: video_stream
Outpue:communication queue Q_video

```
Init UDP Socket bind local_ip and local_port
 Or init V4L2 VideoCapture
if UDP mode then
    send trigger signal b'\x31' to FPGA
data_buffer ← empty bytearray
while True do
    data ←recvfrom(65536)
     Or ret,frame ←cap.read()
    if UDP mode then
        data_buffer.extend(data)
        idx ← data_buffer.find(FRAME_HEADER)
        while idx ≠ -1 do
            if len(data_buffer) ≥ idx + 8 + 614400 then
                frame_bytes ← data_buffer[idx+8 : idx+8+614400]
                data_buffer ← data_buffer[idx+8+614400 :]
                try Q_video.put_nowait(frame_bytes)
                    catch queue.Full:
                        Q_video.get_nowait()
                        Q_video.put_nowait(frame_bytes)
                idx ← data_buffer.find(FRAME_HEADER)
            else
                break
        if len(data_buffer) > 2 × FRAME_BYTES then
            data_buffer.clear()
    else
        try Q_video.put_nowait(frame)
            catch queue.Full:
                Q_video.get_nowait()
                Q_video.put_nowait(frame)
```

进程 2：串口坐标接收与识别辅助线程

该线程运行在 udp_fpga_serial.py 中，与主进程通过锁保护的共享字典协作。线程阻塞等待 FPGA 串口发送的 17 字节坐标帧，收到坐标后读取主线程缓存的最新视频帧，对坐标区域进行外接矩形裁剪、自适应亮度增强，再调用识别模型完成 CTC 文字解码和颜色分类。识别结果写回共享字典，供主进程读取并绘制标注。

Process 2:serial_loop

Input: Serial object **ser**,shared state dict **shared**, thread lock **Lock**,
recognition RKNN model **rec_rknn**
Output: Updated **shared['plate_result']**

```
1.  while not shared['stop'] do
2.      coords <- read_plate_coords(ser)
3.      Lock.acquire()
4.      bgr_img <- shared['latest_frame']
5.      Lock.release()
6.      if bgr_img is None then continue
7.      crop_img <- bgr_img[min_y:max_y, min_x:max_x]
8.      crop_img, enhanced, exposure <- enhance_plate(crop_img)
9.        plate_text, color, rec_conf, color_conf <- recognize_rknn(rec_rknn,
crop_img)
10.     result <- {
          'plate_no': plate_text, 'plate_color': color,
          'rec_conf': rec_conf, 'landmarks': coords,
          'frame': bgr_img.copy(),
          ...
                }
11.     Lock.acquire()
12.     shared['plate_result'] <- result
13.     Lock.release()
```

进程 3：主进程推理与渲染循环

主进程根据运行模式执行不同的推理流水线。

模式 A 加载车牌检测和识别两个 RKNN 模型，依次完成车牌检测、透视校正、亮度增强、CTC 文字识别和颜色分类。支持 UDP、HDMI 两种输入源。绘制结果后输出至 Framebuffer。

模式 B 跳过检测模型，仅加载识别模型，由 FPGA 串口直接提供车牌四角坐标，省去 NPU 检测环节，识别线程根据坐标裁剪 ROI 后直接送入识别模型，显著提升处理速度。

模式 C 加载单个 YOLOv5 模型同时检测行人、红绿灯，斑马线等目标，采用跳帧优化，中间帧复用上次检测结果以降低计算负载，支持 UDP、HDMI 和本地视频三种输入源。绘制结果后输出至 Framebuffer。

Process 3: Main Process Inference and Rendering Loop

Input: IPC queue **Q_video**, shared state dict **shared**, thread lock **Lock**
Output: Framebuffer display output

Mode A —udp_fpga_ai.py, detection and recognition

```
Load det_rknn (plate detection model) and rec_rknn (plate recognition model)
hdmi <- FastFBDisplay("/dev/fb0")
while True do
    raw_frame <- Q_video.get(block=True)
    bgr_img <- decode_rgb565_big_endian(raw_frame)
    det_results <- detect(det_rknn, bgr_img, 320)
    plate_results <- []
    for each det_result in det_results do
        plate_results.append(process_plate(bgr_img, det_result, rec_rknn))
    result_img, result_str <- draw_result(bgr_img, plate_results)
    hdmi.show(result_img)
```

Mode B —udp_fpga_serial.py, display and coordination

```
rec_rknn <- load RKNN recognition model
hdmi <- FastFBDisplay("/dev/fb0")
shared <- {'latest_frame': None, 'plate_result': None, 'stop': False}
Launch Algorithm 2 serial_loop thread
while True do
    raw_frame <- Q_video.get(block=True, timeout=1.0)
    bgr_img <- decode_rgb565_big_endian(raw_frame)
    Lock.acquire()
    shared['latest_frame'] <- bgr_img
    result <- shared['plate_result']
    Lock.release()
    if result is not None then
        display_img <- draw_result(bgr_img.copy(), [result])
    else
        display_img <- bgr_img
    hdmi.show(display_img)
```

Mode C —people_redlight.py

```
det_model <- load RKNN model (runlight_yolov5_640x360.rknn, 8-class YOLOv5)
detector <- RKNNDetector(det_model, conf_thres=0.35, iou_thres=0.45)
hdmi <- FastFBDisplay("/dev/fb0")
frame_queue <- Queue(maxsize=1)
if src == UDP then launch udp_receiver_process(frame_queue)
else if src == HDMI then launch hdmi_receiver_process(frame_queue)
else cap <- VideoCapture(local_file)
frame_count <- 0, last_detections <- []
while True do
    if src == local then
```

```
11.             ret, bgr_img <- cap.read()
12.             if not ret then break
13.         else
14.             raw <- frame_queue.get(block=True)
15.             bgr_img <- decode(raw) or bgr_img <- raw
16.         if skip == 0 or frame_count mod (skip + 1) == 0 then
17.             detections <- detector.infer(bgr_img)
18.             last_detections <- detections
19.         else
20.             detections <- last_detections
21.         result_img, counts <- detector.draw(bgr_img, detections)
22.         hdmi.show(result_img)
23.         frame_count <- frame_count + 1
```

Web 管理平台架构

软件框架

rk3568_manager/	
├── app.py	Flask 主应用，注册路由和 WebSocket
├── config.py	全局配置
├── requirements.txt	Python 环境依赖
├── backend/	后端核心逻辑
│ ├── auth.py	JWT 认证
│ ├── ssh_manager.py	SSH 远程指令管理器
│ └── socket_handlers.py	WebSocket 实时通信处理
├── db/	SQLite 数据库存储目录
└── frontend/	前端 Vue 项目目录
└── index.html	Vue3 单页面应用入口

表 4 结构目录

Web 远程管理平台 rk3568_manager 部署在上位机，为用户提供基于浏览器的远程控制界面，支持远程启停 RK3568 上的推理功能并实时查看运行日志。

平台管理三种推理模式，每种模式对应 RK3568 上的一个入口脚本：

车牌检测和识别，执行 udp_fpga_ai.py

串口坐标识别，执行 udp_fpga_serial.py

行人闯红灯检测，执行 people_redlight.py

同一时刻只允许运行一种模式。

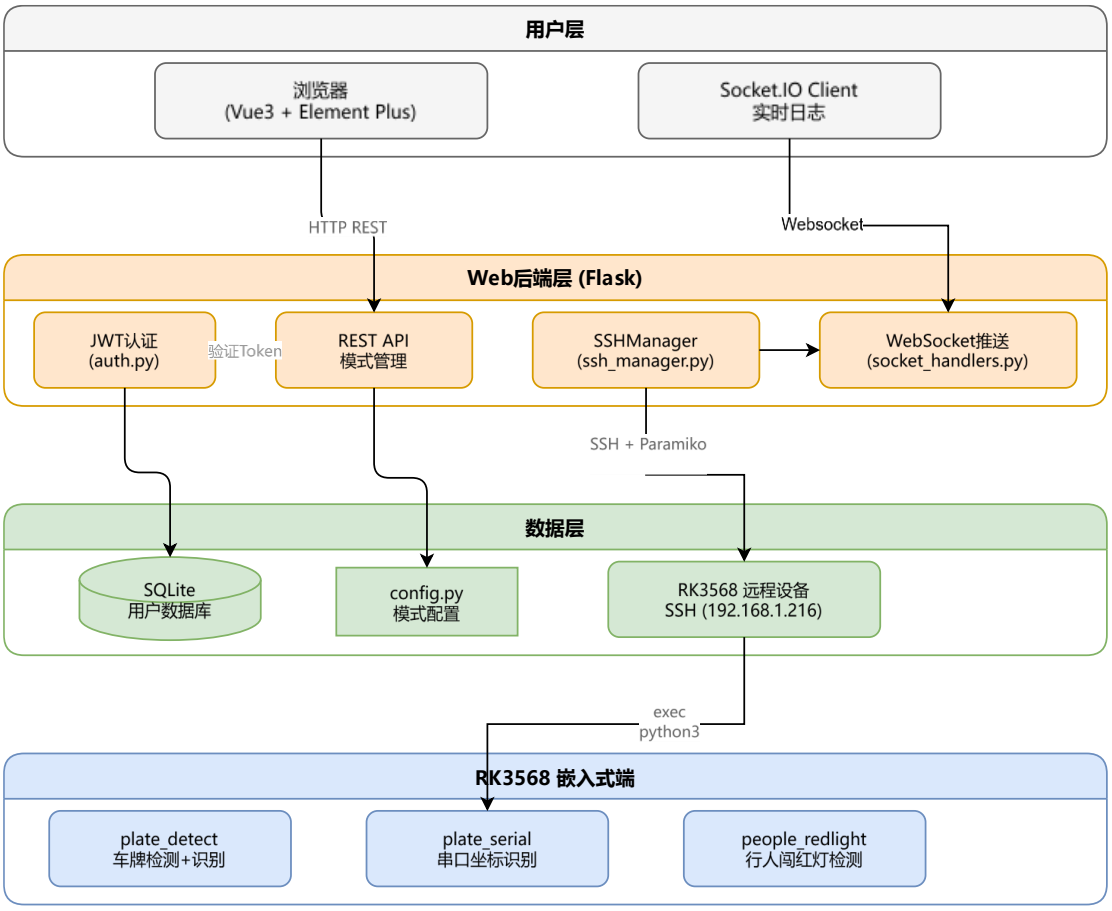


图 11Web 端架构

后端架构：由四个核心模块组成。

app.py：　　Flask 主应用入口，负责注册路由蓝图、配置 JWT 认证中间件、挂载 CORS 跨域支持，并启动 Flask-SocketIO 服务。

auth.py：　　实现 JWT 认证蓝图，提供用户注册、登录和用户信息查询三个接口，用户数据以 bcrypt 哈希存储在 SQLite 数据库中。

ssh_manager.py： 建立 SSH 连接到 RK3568 设备为每个推理模式分配 PTY 终端并执行对应的 Python 脚本，同时在后台线程持续读取输出，通过 SocketIO 推送到前端。

socket_handlers.py：注册 WebSocket 事件处理器，连接时验证 JWT Token，支持模式状态变更通知和实时日志流推送两个事件。

前端架构：单文件 SPA，使用 Vue 3 Composition API 和 Element Plus 2.7.0 构建。包含登录页和管理控制台两个页面。

控制台顶部显示导航栏，左侧展示三个模式卡片，右侧为实时终端日志面板。



图 12 登录页展示

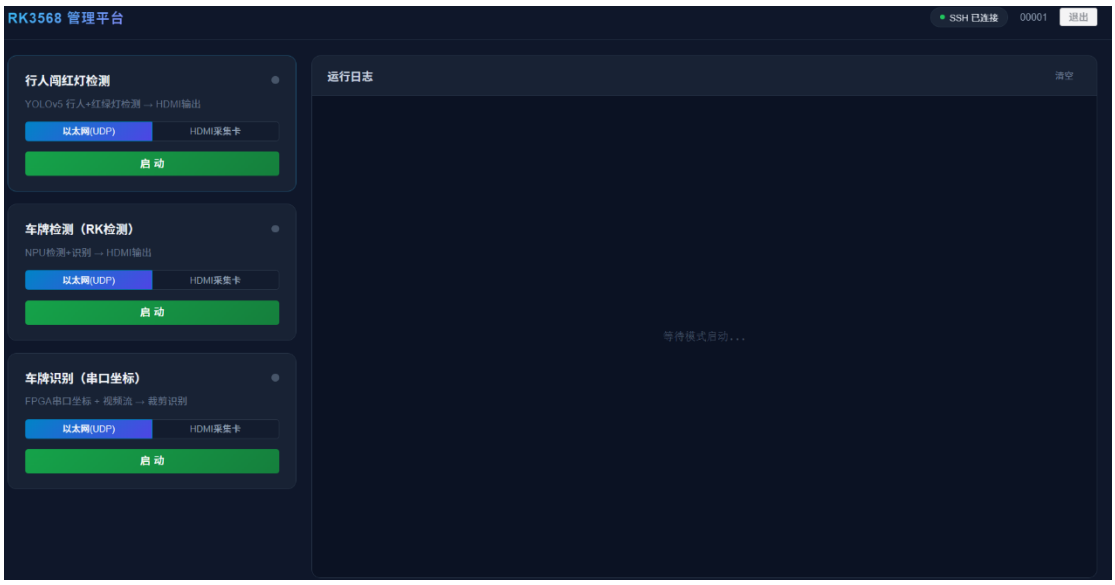


图 13 设备管理界面

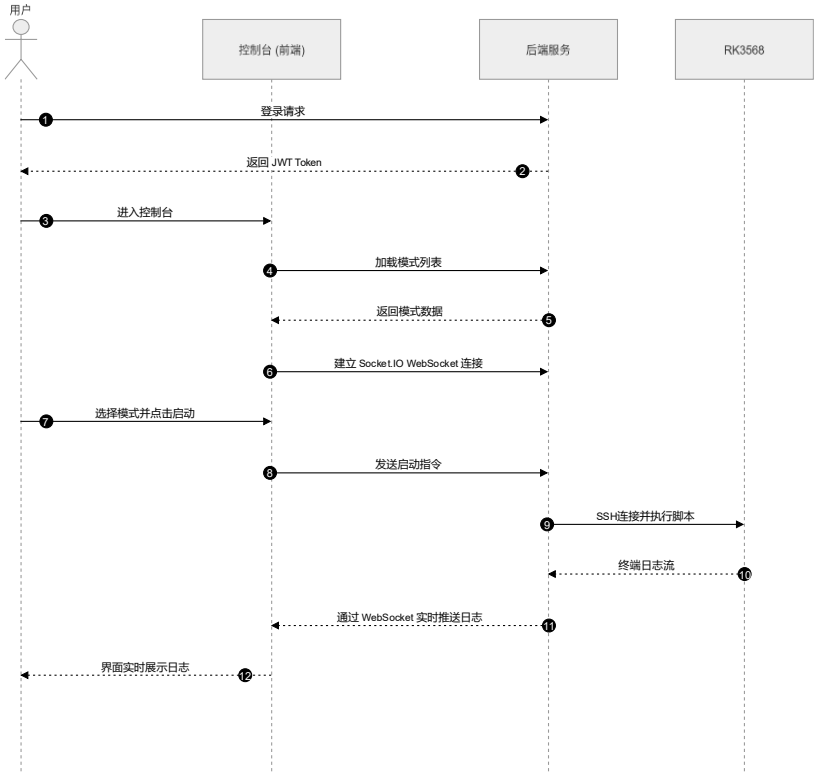


图 14 交互流程

底层核心模块设计

core/fpga_io.py：FPGA UDP 视频流接收

功能：独立进程接收 FPGA 通过 UDP 发送的 RGB565 视频帧，处理粘包问题，将最新帧放入共享队列。

参数	数值	说明
FPGA 地址	192.168.1.10:61440	FPGA 设备 IP 和端口
RK3568 地址	192.168.1.102:61440	本地接收地址和端口
帧头标记	b\xf0\x5a\xa5\x0f	4 字节帧头
帧分辨率	640×480	
帧数据大小	614400 字节	
头部偏移	8 字节	帧头+分辨率信息

表 5 通信参数

触发机制：进程启动后向 FPGA 发送 1 字节信号 0x31，FPGA 收到后开始持续发送视频帧。

粘包处理算法：由于 UDP 是无连接协议，多个数据包可能合并到达。模块使用 bytearray 缓冲区，通过搜索帧头标记定位帧边界，当缓冲区中数据长度足够时截取完整帧数据。缓冲区过大时自动清空，防止内存泄漏。

RGB565 解码：将大端 RGB565 字节流通过 NumPy 视图重组后调用 OpenCV 转换为 BGR 图像。

core/hdmi_io.py：HDMI 采集卡视频流

功能：独立进程通过 V4L2 接口从 HDMI 采集卡采集视频帧。

使用 OpenCV 的 VideoCapture 配合 V4L2 后端，以 MJPG 编码格式从/dev/video9 设备采集 1280×720@30fps 的视频流。MJPG 编码相比原始 YUV 格式大幅降低 USB 带宽需求。采集的 BGR 帧同样通过单槽队列传递给主进程。

core/serial_io.py：串口通信

功能：通过 UART 串口接收 FPGA 发送的车牌四角坐标数据。解析过程为逐字节扫描帧头 0xAA，读取 16 字节数据后解析为 8 个 uint16 值，重组为 4×2 的坐标矩阵。

core/display.py：HDMI 显示输出

功能：通过 Linux Framebuffer 实现零拷贝 HDMI 视频输出。

FastFBDisplay 类初始化时读取/sys/class/graphics/fb0/下的分辨率和色深参数，使用 mmap 将帧缓冲区映射到用户空间。将 OpenCV 图像缩放后转换为对应色深格式，直接通过内存拷贝写入帧缓冲区，避免系统调用开销。支持 16bpp 和 32bpp 两种色深。

core/postprocess.py：YOLO 后处理

功能：NumPy 实现的 YOLOv5 特征图解码和 NMS 后处理。

YOLOPostProcessor 类接收模型的 3 个输出特征图，通过 sigmoid 处理横纵坐标偏移和置信度，应用锚框解码公式还原为实际坐标。解码输出包含边界框、置信度、4 个 landmark 点坐标和类别概率。使用 grid_cache 字典缓存网格坐标避免重复创建。

NMS 使用 OpenCV C++后端的 cv2.dnn.NMSBoxes 加速执行。坐标还原函数将结果从 letterbox 预处理坐标系映射回原始图像坐标系。

core/image_utils.py：图像工具

提供通用图像预处理函数：letterbox()保持宽高比缩放并用灰色边框填充；clip_boxes()将边界框裁剪到图像范围内。

车牌检测识别与行人检测模块[5]

plate_det/plate_detector.py：车牌检测器

使用 YOLOv5 模型通过 RKNN NPU 推理实现车牌检测。

模型文件：plate_detect_320_int8.rknn

检测流程：输入图像经 letterbox 缩放至 320×320→BGR 转 RGB→转为 NCHW 格式→RKNN 推理→YOLOPostProcessor 解码特征图 →NMS 过滤→坐标还原到原图尺度。每个检测结果包含：边界框坐标、置信度、用于车牌透视校正的 4 个角点 landmark 坐标和类别标签。

plate_det/plate_recognizer.py：车牌识别器

对裁剪后的车牌 ROI 图像进行文字识别和颜色分类。

模型文件：plate_rec_color_fixed.rknn

CTC 解码：取每步 argmax，去除重复和空白符，查 78 字符字典 plateName 映射为最终文字。字典包含 31 个省份简称、字母 A-Z、数字 0-9 及特殊字符学、警、港、澳、挂、使、领、民、航。

颜色分类：softmax 后取最大概率类别，共 5 类：黑色、蓝色、绿色、白色、黄色。

图像增强：enhance_plate()根据灰度均值判断曝光状态。均值>200 判定为过曝，降低亮度，均值<80 判定为欠曝，提高亮度和对比度，正常则不处理。

plate_det/plate_pipeline.py：处理流水线

process_plate()函数整合完整的车牌处理流程：

1.校正车牌为正视图。

2.亮度增强，调用 enhance_plate()自适应调节。

3.双层车牌处理：当检测类别为双层车牌时，调用 get_split_merge()将上下两层水平拼接为单行，便于识别。

4.RKNN 识别：调用 recognize_rknn()获得文字和颜色。

plate_det/plate_draw.py：可视化绘制

draw_result()在原图上绘制红色矩形框标记车牌位置，并在框旁绘制中文文字标签。由于 OpenCV 不支持中文渲染，使用 PIL/Pillow 的 ImageFont.truetype 加载中文字体，通过 NumPy 数组与 PIL Image 互转实现绘制。

people_det/people_detector.py：行人闯红灯检测器

基于 RKNN 的 YOLOv5 模型，检测行人，红绿灯，斑马线等多个目标。

RKNNDetector 类封装完整的推理流程。

推理流程：letterbox 缩放→RGB 转换→NPU 推理→按类别分别执行 NMS→坐标还原。

绘制方法：闯红灯目标用红色矩形框，普通行人用绿色矩形框，返回标注后图像和统计计数。

people_det/people_run.py：多模式运行器

提供三种运行模式，均支持跳帧优化：

模式	函数	输入源	说明
FPGA 以太网	run_fpga()	UDP 视频流	RGB565 解码→检测→HDMI 输出
HDMI 采集卡	run_hdmi()	HDMI	BGR 帧→检测→HDMI 输出
本地视频	run_video()	视频文件	支持 HDMI 输出和窗口预览

表 6 模式运行说明

2.3.3 FPGA 端算法模块介绍

FPGA 以太网传输模块

video_udp_pkt

设计功能：模块跨时钟域 cam_pclk 到 eth_tx_clk 处理摄像头 16 位像素流，通过异步 FIFO 将相邻两像素拼接为 32 位数据；在帧同步下降沿写入 32 位帧头与分辨率字作为首包前缀，后续各包仅为图像数据；当 FIFO 内数据量达到首包 H×2+8 字节或后续包 H×2 字节的长度时，触发 UDP 发送请求，外部模块响应读取即可完成组包；帧结束或停止传输时立即复位 FIFO，避免帧间数据串扰。

helai_udp

设计功能：模块实现 UDP 硬件协议栈，通过 GMII 接口与外部 PHY 交互，完成 UDP 数据包的接收与发送。接收侧从以太网帧中解析出 UDP 负载，输出数据、有效字节数及接收完成标志；发送侧根据用户提供的目标 MAC、IP 及端口，自动封装 UDP/IP 首部，并通过 CRC32 模块生成帧尾校验，在外部请求信号控制下发送数据。模块参数化配置本机及对端的 MAC、IP 和端口号，便于移植适配不同网络。

ikun_gmii_2_rgmii

设计功能：模块实现 RGMII 与 GMII 接口的双向转换：接收侧将 RGMII 的 DDR 数据 rgmii_rd、rgmii_rx_ctl 通过 ISERDES 转换为 GMII 的 8 位并行数据及控制信号，并输出 gmii_rx_clk、gmii_rx_dv、gmii_rx_er 和 gmii_rxd；发送侧将 GMII 的发送数据 gmii_txd、gmii_tx_en、gmii_tx_er 经 OSERDES 转换为 RGMII 的 DDR 信号输出，同时利用 PLL 相移和 IODELAY 调整发送时钟 rgmii_txc 以满足时序要求。支持 10/100/1000Mbps 速率及全/半双工模式。

顶层模块函数流程图：

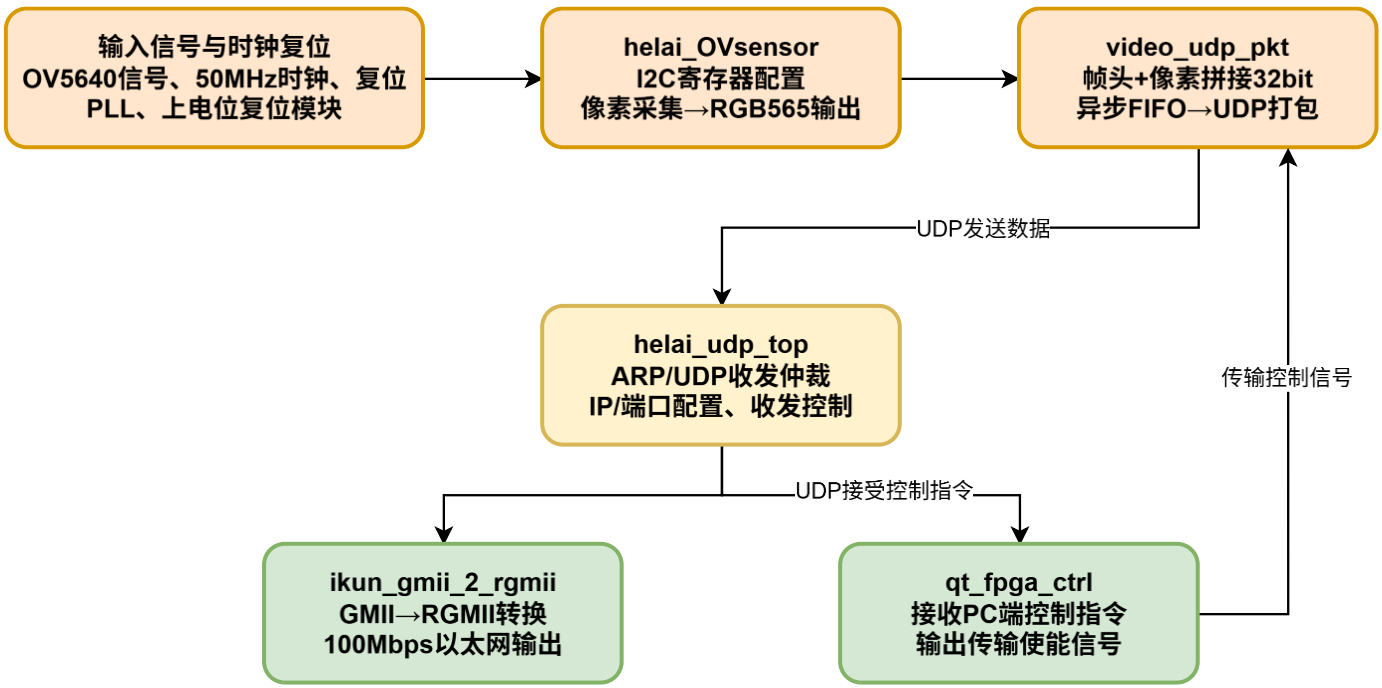


图 15 以太网传输顶层模块流程图

主要输入输出接口：

信号名称	方向	功能说明
i_clk_50m	input	系统主时钟 50MHz
rst_n	input	全局复位，低电平有效
cmos1_scl	output	OV5640 I2C 时钟
cmos1_sda	inout	OV5640 I2C 数据
cmos1_vsync	input	OV5640 场同步
cmos1_href	input	OV5640 行有效
cmos1_pclk	input	OV5640 像素时钟
cmos1_db[7:0]	input	OV5640 图像数据
cmos1_rstn	output	OV5640 复位
cmos1_pwdn	output	OV5640 掉电控制
rgmii_rxc	input	RGMII 接收时钟
rgmii_rxctl	input	RGMII 接收控制
rgmii_rxd[3:0]	input	RGMII 接收数据
rgmii_txc	output	RGMII 发送时钟
rgmii_txctl	output	RGMII 发送控制
rgmii_txd[3:0]	output	RGMII 发送数据
phy_rst_n	output	以太网 PHY 复位

表 7 以太网传输输入输出接口

FPGA 车牌识别框选模块

主要功能函数设计说明

RGB2YCbCr

RGB2YCbCr 模块实现 RGB565 图像格式到 YCbCr 颜色空间的转换，作为图像处理管线中的第一个处理单元。该模块接收 HDMI 输入的 RGB565 像素数据，将其转换为 YCbCr 颜色空间，以便于后续的图像处理操作，特别是提取亮度(Y)分量与后续 HSV 处理结合用于筛选车牌颜色。

函数流程图：

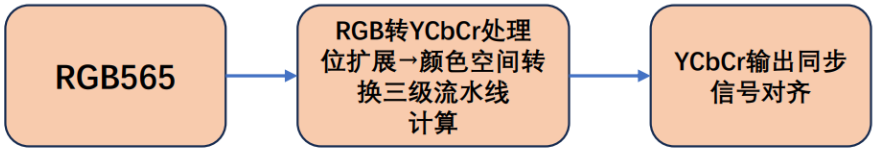


图 16 YCbCr 流程图

RGB2HSV

RGB2YCbCr 模块实现 RGB 到 HSV 色彩空间转换，即根据输入的 24bit RGB 分量计算出对应的 9bit 色调、8bit 饱和度和 8bit 明度。在顶层模块与 YCbCr 的颜色掩码 is_blue_ycbcr 相结合，双重颜色验证，提高在复杂光照下对车牌的检测准确性。只有同时满足 YCbCr 和 HSV 条件，才被认为是有效的对应车牌颜色像素，从而进入后续的二值化、形态学处理和角点检测流程。

函数流程图：

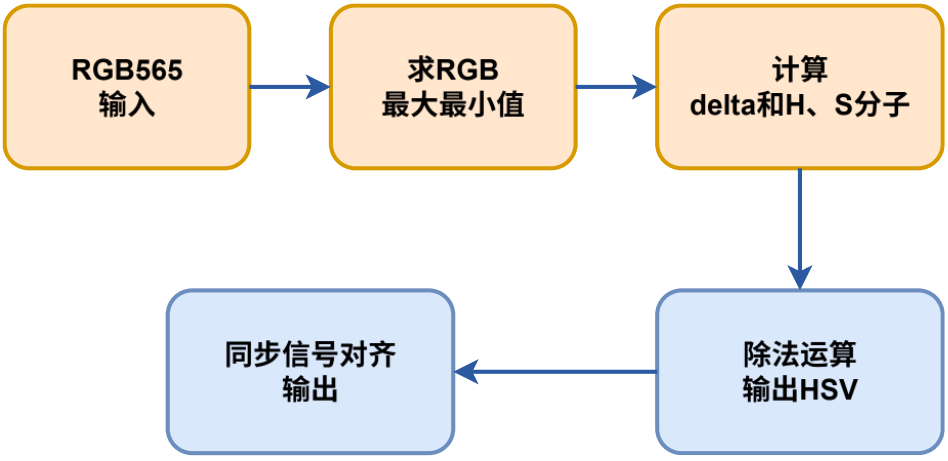


图 17 HSV 流程图

Binarization

Binarization 模块实现图像的二值化处理，将灰度图像 Y 转换为黑白二值图像。通过可调的阈值参数，将亮度值高于阈值的像素设为白色，低于阈值的像素设为黑色，阈值由顶层对应识别车牌颜色赋值对应。

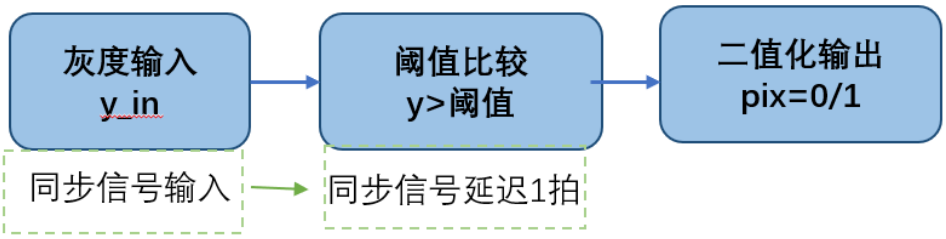


图 18 二值化流程图

median_filter_3x3

median_filter_3x3 模块实现了一个 3×3 中值滤波器，用于图像降噪处理。该模块通过对输入的 3×3 矩阵中 9 个像素进行排序，找出中值像素，有效去除图像中的椒盐噪声，同时保留图像边缘细节，为后续的二值化和车牌检测更清晰的图像数据。

函数流程图：

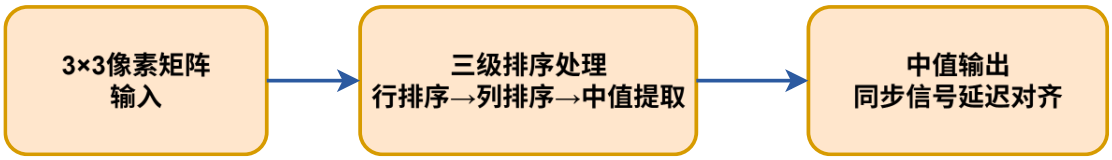


图 19 3×3 中值滤波流程图

Bit_Erosion_Detector

Bit_Erosion_Detector 模块实现二值化图像 3×3 形态学腐蚀处理，对预处理后的单通道二值化图像进行腐蚀运算。模块通过矩阵生成单元实时生成连续图像的 3×3 像素窗口，随后对窗口内所有 9 个像素进行多级逻辑与运算。只有当 3×3 窗口内全部像素均为有效目标像素时，当前中心像素才判定为有效，否则置为背景。在特定颜色要求的二值化基础上收敛白色目标边缘、滤除图像中小面积白色噪点、断开细小粘连干扰，为后续膨胀处理、边缘检测与目标轮廓提取提供更干净的图像数据。

函数流程图：

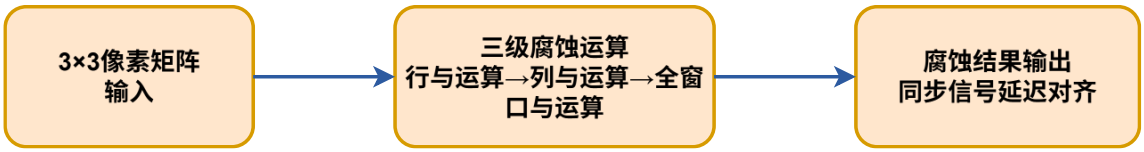


图 20 腐蚀流程图

Bit_Dilation_Detector

Bit_Dilation_Detector 模块基于二值化图像实现形态学的膨胀，对输入分辨率二值化图像执行 3×3 窗口形态学膨胀运算，通过在顶层实例化三次，输出膨胀后的二值图像，同时严格同步图像的场同步、行同步、像素时钟使能等控制信号。

函数流程图：

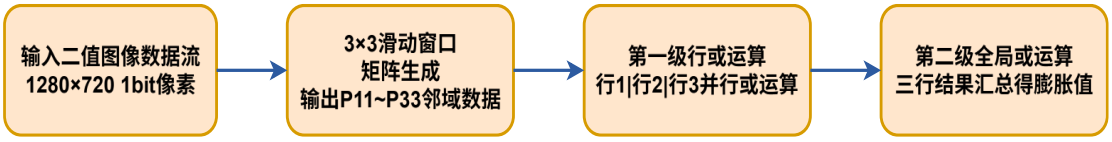


图 21 膨胀流程图

Bit_HDilation

Bit_HDilation 模块是水平方向一维膨胀算子模块。模块对膨胀半径进行设置，容忍有一定倾角的二值化模块输入。对输入的二值图像执行水平方向的形态学膨胀运算，实现水平方向扩张、微小空洞填补、断裂区域连接等效果。用于对车牌区域横向膨胀，更好的适配形状识别要求。

函数流程图：

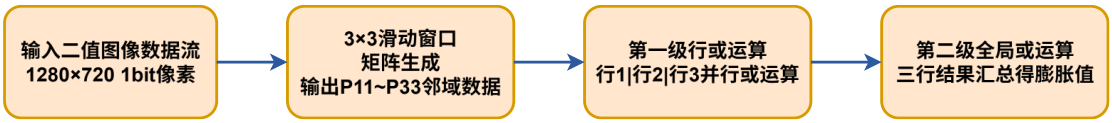


图 22 水平膨胀流程图

Frame_buf

frame_buf 模块实现图像帧缓存功能，通过 DDR 内存实现图像数据的存储和读取。该模块将输入图像数据写入 DDR 内存，然后按照输出时序读取数据显示，支持不同分辨率的图像处理和显示。

函数流程图：



图 23 帧缓存流程图

super_plate_tracker

设计功能：super_plate_tracker 模块接收到上层形态学处理后的二值图像流与像素坐标、行场同步信号，通过双路独立追踪寄存器组构建目标势力范围，在限度内容忍断裂间隙完成连通域融合；同时实时动态更新目标上下左右边界，利用像素坐标和、差值极值算法确定车牌四边形四角坐标，统计有效白点数量结合目标面积完成长宽比、像素密度筛选。针对与新能源的绿牌车辆，在满足形态学长宽特征、颜色筛选的情况下做一定比例的向上偏移矫正。单帧结束后对双目标进行有效性判定，最终输出最多两组车牌左上、右上、左下、右下四角坐标及有效标志。

函数流程图：

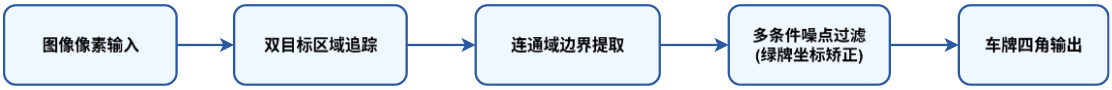


图 24 车牌框选流程图

顶层模块函数流程图：

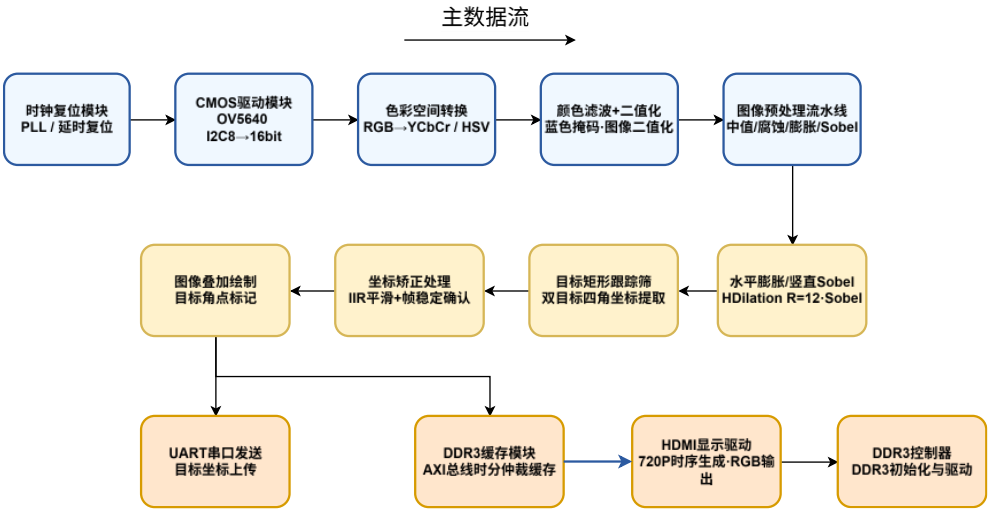


图 25 车牌识别顶层流程图

主要输入输出接口：

信号名称	方向	功能说明
sys_clk	input	系统主时钟，驱动 PLL / UART / DDR 参考
clk_p / clk_n	input	供 DDR3 使用差分输入时钟
cmos1_scl / sda	inout	I2C 总线，配置摄像头寄存器
cmos2_vsync / href / pclk	input	帧同步、行参考、像素时钟
cmos2_data[7:0]	input	8 位原始像素数据
cmos2_reset	output	摄像头硬件复位控制
mem_ck / mem_ck_n	output	DDR3 差分时钟
mem_a[14:0]	output	行地址
mem_ba[2:0]	output	Bank 地址
mem_dq[31:0]	inout	数据选通差分信号
uart_tx	output	UART 发送，每帧末将定位坐标上报 RK
ddr_init_done	output	DDR3 初始化完成标志，低时帧缓存禁读写

表 8 车牌识别输入输出接口

行人闯红灯预警检测模块设计

色彩空间预处理模块

复用车牌识别的色彩预处理模块。

dynamic_pedestrian_detector

设计功能：dynamic_pedestrian_detector 模块通过逐帧特定区域的颜色判定进行红绿灯状态的确定；运用帧差法，判定运动行人，将输入亮度的高 4 位存入 BRAM，延迟一帧，计算当前 4 位亮度与上一帧同位置亮度差的绝对值 abs_diff，累加整帧所有 abs_diff，帧结束时取平均的高 4 位作为 global_bias，对每个像素的 abs_diff 减去 global_bias 并对结果进行非负截断，得到补偿后的差分 abs_diff_comp。判定 abs_diff_comp>3 且有效数据有效时生成最终的运动信号。模块内部嵌入了腐蚀、膨胀，同时动态记录最小/最大 x、y 坐标，最终在帧结束时得到原始运动矩形框。为避免错框多框，模块叠加宽高比限制、连续帧确认及 IIR 防抖逻辑。

函数流程图：

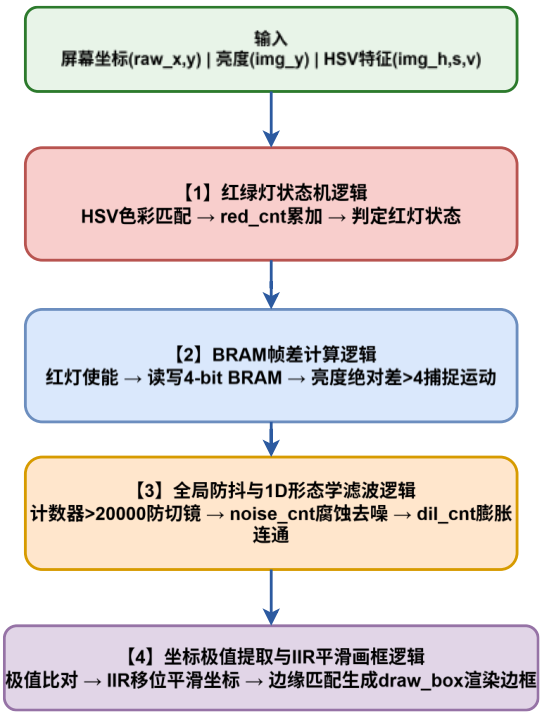


图 26 帧差模块流程图

顶层模块流程图

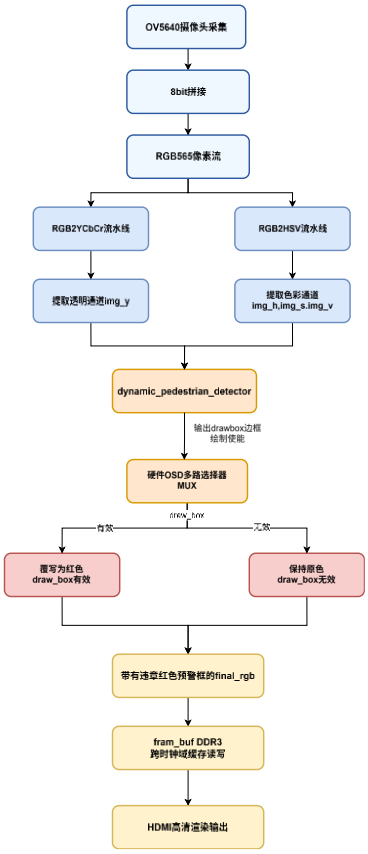


图 27 行人闯红灯顶层模块流程图

3 性能参数

3.1 FPGA 端车牌检测系统误差分析

误差来源：由于 FPGA 部分进行了完整的预处理链包括：RGB565 截断、RGB→YCbCr/HSV 转换、蓝色车牌的双重颜色掩码、固定阈值二值化、3×3 中值滤波、腐蚀×1、膨胀×3、半径为 12 的水平膨胀，以及最终基于连通域追踪且包含对角线极值、密度校验、绿牌补偿等处理的 super_plate_tracker，会出现：

- 量化与数值近似：YCbCr/HSV 定点计算时 FPGA 用整数乘法和移位代替浮点，产生舍入误差；HSV 中的除法截断导致色调误差±1°；同时，送入 HSV 模块时的 {r[4:0], r[4:2]} 方式并非精确 8 位值，引入系统性偏差。
- 形态学操作的几何副作用：腐蚀操作中，3×3 AND 操作使车牌向内收缩，边界内移；水平半径为 12 膨胀 3 次时，3×3 OR 和水平长条结构元素使车牌向外扩张，累积外扩可达数十像素，产生系统性偏移；中值滤波虽能去噪，但会轻微平滑角点，引入亚像素级误差。
- 算法原理性误差：Tracker 对角线极值法中用 x+y 极值定位 TL/BR、x-y 极值定位 TR/BL，在车牌倾斜时这些极值点并非真正的角点，原理上即存在偏移，倾斜角越大误差越大；由于 25%密度校验阈值的设定，像素数×4 >面积的判断条件在目标被轻微遮挡或噪声稀疏时可能判定无效，造成漏检。
- 空间离散化与扫描顺序：连续坐标到整数像素的舍入，以及光栅扫描顺序对连通域合并的影响，都会引起±0.5 像素的随机误差。

由此我们引入 python 代码对预处理和实际 tracker 模块的算法进行模拟，按照实际 FPGA 工程对应的处理逻辑比如 3×3 矩阵实现中值滤波、膨胀等具体逻辑，遍历一百多个独立车牌图像和测试案例，设计了 6 组对照实验，对不同角度、噪声强度的合成车牌图像，覆盖倾斜角 0~20°、噪声 0~40DN，完成系统误差的完整量化。具体主要实验如下：

实验编号	实验名称	图片数量	说明
A	形态学边界偏移	1	固定参数，不循环
B	倾斜角	11	角度从 0°到 20°，步长 2°
C	噪声影响	40	10 个噪声级别×4 次重复
D	颜色窗口扫描	52	B 分量从 100 到 255，步长 3
F	误差分解	1	固定标准条件

绘制出每个形态学处理阶段的边界扩展各个对照试验结果展示如下：

- 0 度，标准光条件下的检测结果和 12 度倾斜条件下的检测结果

展示了系统实际检测效果。绿色虚线框是真实车牌位置，红色实线框是 Tracker 输出的预测框，框的四角标有 TL/TR/BL/BR 四个角点。0 度正置时红框与绿框基本重合，说明在标准条件下检测精度良好。12 度倾斜时两个框依然接近，说明 Tracker 的对角线极值法对轻度倾斜有较好适应性。但可以观察到红框比绿框明显偏大，这正是形态学膨胀引入的系统性外扩误差。

- 每个形态阶段的边界扩展

柱状图定量显示了每个形态学阶段相对真实车牌边界的外扩量，分左右上下四个方向。关键结论是水平膨胀 hdilation 阶段的左右各外扩约 14px，远大于其他阶段，是误差的单一最大来源。腐蚀阶段略有内缩，三次膨胀各外扩 2px，叠加后净外扩约 2px，水平膨胀再额外加 14px。在二进制流水线阶段，纵向拼接了五个阶段的二值图。从上到下可以看到：mask 阶段颜色掩码刚刚提取出蓝色区域，median 阶段去除孤立噪点后形状更干净，erosion 阶段略微收缩，dilation3 阶段向外膨胀填充，hdilation 阶段水平方向显著拉宽，这直观解释了为何水平误差远大于垂直误差。

- 倾斜角与角位误差

横轴是倾斜角 0-20 度，纵轴是平均角点误差。代表从 Mask 直接到 Tracker 且无形态学处理的蓝线始终接近 0px，说明 Tracker 本身的对角线极值算法对倾斜不敏感，误差极小。代表完整流水线的红线稳定在约 13.4px 且几乎不随角度变化，说明这 13px 误差完全来自形态学外扩，与倾斜角度无关。绿色填充区域就是形态学附加的误差量，恒定约 13px。

- 噪声与角落误差和 D：噪声与检测率

左图显示噪声 σ=0 时来自形态学的误差约 13px，随着噪声增大误差急剧上升到 70px。右图显示检出率在 σ=8 时跌破 50%，σ=10 之后几乎完全失检。说明系统抗噪能力很弱，σ 超过 8 DN 就会大量漏检。

- 彩色遮罩窗口（R=30，G=170，B=扫描）

固定 R=30、G=170，扫描 B 值从 100-255，观察蓝线 Cb 值和红线 H 值的变化，以及最终是否通过二值化（绿色阶梯）。可以看到只有 B 值在约 230~255 范围内即右侧绿色阶梯跳到 1 的区域时才能同时满足 Cb>135 和 H≥200°，通过窗口极窄。黄色虚线标注了 Cb=135 的阈值，橙色点线标注了 H=200°的边界，清楚地显示出两套判据共同形成的严格联合约束。

- 系统错误量化

将所有误差来源按行列出，最后一行橙色高亮的 TOTAL 行给出了最终结论：水平方向系统误差约 28px，均值角点误差 13.4px，垂直约 4px，16 帧仅有 1 帧检出。由形态学外扩引起的可校正的系统性误差可通过固定偏移量补偿消除，真正难以解决的是颜色掩码窗口过窄导致的高漏检率，需要改为自适应阈值才能从根本上改善。



图 28 车牌检测系统误差分析

3.2 FPGA 资源开销

Module Inst Name	LUT	FF	Distributed RAM	APM	DRM	HSSTLP	IO	USCM
hdmi_ddr_ov5640_top	8070	7815	39	3	15	0	131	2
cfg_pll_inst	0	0	0	0	0	0	0	0
cmos2_8_16bit	5	82	0	0	0	0	0	0
coms1_reg_config	42	34	0	0	0.5	0	0	0
u1	20	9	0	0	0	0	0	0
coms2_reg_config	31	23	0	0	0.5	0	0	0
u1	20	9	0	0	0	0	0	0
fram_buf	376	629	0	0	8	0	0	0
rd_buf	62	94	0	0	4	0	0	0
wr_buf	189	370	0	0	4	0	0	0
wr_rd_ctrl_top	125	165	0	0	0	0	0	0
ms72xx_ctl	210	271	0	0	1	0	0	0
U1_ms7200_ctl	68	144	0	0	0.5	0	0	0
U2_ms7210_ctl	71	63	0	0	0.5	0	0	0
iic_dri	61	61	0	0	0	0	0	0
power_on_delay_inst	42	36	0	0	0	0	0	0
sync_vg	49	25	0	0	0	0	0	0
u_Dilate_1	21	62	0	0	1	0	0	0
u_matrix_generate_3x3_1bit	16	54	0	0	1	0	0	0
u_Dilate_2	21	62	0	0	1	0	0	0
u_matrix_generate_3x3_1bit	16	54	0	0	1	0	0	0
u_Dilate_3	20	62	0	0	1	0	0	0
u_matrix_generate_3x3_1bit	16	54	0	0	1	0	0	0
u_Erosion	21	62	0	0	1	0	0	0
u_matrix_generate_3x3_1bit	16	54	0	0	1	0	0	0
u_HDilation	18	52	0	0	0	0	0	0
u_VIP_Gray_Median_Filter	27	73	0	0	1	0	0	0
u_Median_Filter_3X3	10	13	0	0	0	0	0	0
u_matrix_generate_3x3_8bit	16	54	0	0	1	0	0	0
u_binarization	7	3	0	0	0	0	0	0
u_ddr3_test_h	4614	4946	39	0	0	0	70	2
u_ddrc_rstn_sync	0	2	0	0	0	0	0	0

图 29FPGA 车牌识别资源开销

3.3 RK3568 侧误差测算

3.3.1. 模型训练概述

在车牌检测识别部分系统包含两个独立训练的模型：

- 检测模型：基于 YOLOv5s 改进，输入 320×320 图像，输出车牌边界框及四个角点坐标。损失函数由边界框回归损失、置信度损失、分类损失和关键点损失四部分加权组成。使用 SGD 优化器，余弦退火学习率调度，训练过程中采用 Mosaic 拼接、HSV 色彩抖动、多尺度训练等数据增强策略。

- 识别模型：采用卷积网络与 CTC 损失的端到端架构，输入 168×48 的车牌裁剪图，直接输出字符序列，无需逐字符分割。支持含省份简称、字母、数字及特殊标识的 78 个字符类别。使用 Adam 优化器，阶梯式学习率衰减。

两个模型分别导出为 ONNX 格式。NMS 等后处理部分在模型外部以独立代码实现，不嵌入 ONNX 模型。这是因为 NMS 等涉及动态形状和条件判断的算子在 INT8 量化时精度损失较大，外置后可保证经过 Rknn-Toolkit2INT8 量化后的检测精度，同时便于部署时灵活调整阈值。

3.3.2 实验配置

硬件：RTX 4060，16GB 内存

数据集：CCPD2019，base、blur、challenge、db、rotate 五个子集中随机抽取 60,000 张训练，15,000 张验证。

参数	检测模型	识别模型
训练轮数	50	50
批大小	16	64
初始学习率	0.01	0.001
优化器	SGD	Adam

检测模型训练指标：

轮次	边界框损失	置信度损失	分类损失	精确率	召回率	mAP@0.5	mAP@0.5:0.95
1	0.0382	0.0571	0.0034	0.612	0.483	0.521	0.283
5	0.0247	0.0385	0.0019	0.785	0.692	0.738	0.462
10	0.0189	0.0293	0.0013	0.856	0.781	0.827	0.558
15	0.0156	0.0237	0.0010	0.893	0.832	0.817	0.623
20	0.0134	0.0198	0.0008	0.912	0.861	0.896	0.671
25	0.0118	0.0172	0.0007	0.924	0.879	0.912	0.708
30	0.0106	0.0153	0.0006	0.933	0.891	0.923	0.736
35	0.0097	0.0140	0.0005	0.940	0.901	0.931	0.757
40	0.0091	0.0130	0.0005	0.945	0.908	0.937	0.773
45	0.0086	0.0122	0.0004	0.949	0.914	0.942	0.785
50	0.0082	0.0116	0.0004	0.953	0.918	0.946	0.795

各项损失在前 10 轮快速下降，之后平稳收敛。边界框损失从 0.038 降至 0.008，mAP@0.5 从 52.1%提升至 94.6%，训练过程无明显过拟合现象。

识别模型训练指标：

轮次	训练损失	字符准确率	整牌准确率	学习率
1	2.874	0.423	0.186	0.001
5	1.052	0.712	0.498	0.001
10	0.519	0.836	0.672	0.0005
15	0.325	0.889	0.768	0.0005
20	0.218	0.918	0.831	0.0003
25	0.156	0.937	0.872	0.0003
30	0.113	0.951	0.899	0.0001
35	0.087	0.960	0.918	0.0001
40	0.070	0.967	0.932	0.0001
45	0.058	0.972	0.942	0.0001
50	0.050	0.976	0.950	0.0001

训练损失从 2.87 快速收敛至 0.05，前 5 轮即完成主要学习过程。整牌准确率从 0.186 提升至 0.950，验证集指标与训练集同步收敛，未出现过拟合现象。

3.3.3 模型的量化与部署

量化流程：

量化过程使用 100 张从验证集中随机抽取的校准图像，覆盖不同光照、角度和模糊场景。Rknn-Toolkit2 工具根据校准数据的激活值分布确定每层的量化参数，将浮点权重和激活值映射到 INT8 范围，优化级别设为最高级 Level 3。

后处理 NMS 等过程不参与量化，保持浮点精度独立运行。这一设计避免了 NMS 中动态排序和条件分支算子带来的量化精度损失。

量化精度对比：

模型	指标	ONNX	RKNN	INT8 精度损失
检测	mAP@0.5	0.946	0.935	-0.011
检测	mAP@0.5:0.95	0.950	0.773	-0.022
检测	精确率	0.953	0.942	-0.011
检测	召回率	0.918	0.904	-0.014
识别	字符准确率	0.976	0.968	-0.008
识别	整牌准确率	0.950	0.936	-0.014

检测模型 INT8 量化后 mAP@0.5 下降 1.1 个百分点，识别模型整牌准确率下降 1.4 个百分点，均在可接受范围内。识别模型量化损失更小，原因是其纯卷积结构中算子规则、数值范围稳定，量化适配性更好。

推理性能对比：

在 RK3568 开发板上实测的单帧推理耗时：

处理阶段	ONNX	RKNN
检测推理	200ms	50ms
识别推理	30ms	10ms

INT8 量化后，检测推理加速 3 倍，识别推理加速 2 倍，满足实时部署要求。

本地实际遍历 CBLPRD-330k 数据集 300 张车牌（蓝 100、绿 100、黄 90、白 5、黑 5）准确率在 97%，主要集中在蓝黄颜色识别的准确率上，由于数据集本身图像质量的限制大约五张出现完全识别不到，大概有五张出现黄蓝错识或字符字型出错。

4. 仿真与测试报告

4.1 系统模块功能仿真测试

仿真时为验证重要模块功能时序与逻辑的正确性稳定性，同时避开未开源 IP 包括 DDR 等的逻辑建模，对 FPGA 工作的主要的两大模块的两个核心底层模块进行仿真：以太网传输 UDP 打包异步 FIFO 读入逻辑验证；车牌框选定位模块像素遍历与寄存器功能流程验证。仿真时间精度为 1ns/1ps，仿真激励完全参照实际硬件逻辑参数配置。

4.2 仿真环境与参数设置

仿真采用 PDS 与 ModelSim SE-64 10.4 联合仿真。

4.3 模块仿真结果分析

以太网发送 UDP 打包及 FIFO 异步读写逻辑验证

1.模块功能概述

video_udp_pkt 是以太网发送时的核心模块，负责接收摄像头 CMOS 输出的 16 位图像像素数据；给每帧图像添加**帧头 + 分辨率信息**，方便接收端识别；把数据缓存到**异步 FIFO** 以解决跨时钟域问题；按照 UDP 协议要求**自动组包、发送以太网数据包；实现一帧图像分多包稳定传输到 RK3568。**为验证传输流程及对应逻辑，分别仿真了 **1280×720** 分辨率下和 **640×480** 下的模块，为完整体现行结束包开始包结束等逻辑 **1280×720 仿真四包两行，640×480 仿真一包一行。**

2.仿真验证逻辑

仿真以太网打包及异步 FIFO 写入逻辑，验证核心目标如下：

- 验证帧头、分辨率组包写入逻辑：检测 img_vsync 下降沿 neg_vsync，下降沿瞬间是否写入自定义帧头 IMG_FRAME_HEAD，延迟一拍 neg_vsync_d0 写入分辨率{H,V}，确保 RK 及上位机能识别帧起始、获取分辨率参数。
- 验证 16bit 像素转 32bit 拼接逻辑：利用 wr_sw 交替打拍缓存两个 16 位像素，验证单行 640 像素完整拼包无错位、无丢失，确保数据拼接时序正确性。
- 验证异步 FIFO 跨时钟域读写：cam_pclk 为摄像头域时钟，eth_tx_clk 为以太网域时钟，存在跨时钟亚稳态，须验证摄像头与以太网不同频的跨域缓存问题。
- 验证 UDP 自动组包&发包阈值逻辑：验证模块自动判断缓存够不够、自动触发发包的核心控制逻辑。
- 验证 UDP 握手交互时序。
- 验证帧复位与 FIFO 清空机制。

3.仿真结果分析

仿真过程与预期结果：

- 仿真启动后所有信号置为默认值：rst_n=0、img_vsync=0、img_data_en=0、transfer_flag=0。两个时钟同时运行，cam_pclk 约 84 MHz，eth_tx_clk 125 MHz。经过 10 个 cam_pclk 周期后 rst_n 拉高，再过 2 拍后 transfer_flag=1 使能传输。
- 完整帧消隐期与帧头写入（约 120～500 ns）：CAM 线程拉高 img_vsync=1，等待约 20 个 cam_pclk，模拟帧消隐期（blanking）。neg_vsync 触发帧头写入：wr_fifo_en 变为 1，wr_fifo_data 写 0xF05AA50F (IMG_FRAME_HEAD)。ETH 侧同步：img_vsync_txc_d0/d1 经两拍同步到 eth_tx_clk 域，neg_vsynt_txc 触发 udp_tx_byte_num 更新：udp_tx_byte_num 赋值 640×2 + 8 = 1288 (byte)。
- 行像素写入 FIFO：send_row(0)任务逐拍发送 640 个像素，每次 img_data_en 有效时，wr_sw 翻转，同时 img_data_d0 锁存当前像素。
- ETH 侧检测并读包，DUT 接收到 udp_tx_done，tx_busy_flag 清零，udp_tx_byte_num 更新为 640×2=1280（为下一行准备，但本仿真不会再触发）。ETH 线程置 ev_all_pkt_done=1，通知 CAM 线程。
- 帧结束并执行 FIFO 复位。

4.仿真输出：

1280×720 分辨率与 640×480 仿真逻辑类似，读波形发现写入 FIFO 的顺序及逻辑正确，首包首行写入字节数正确，img_data 与 FIFO_data 字节写入对应验证正确，仿真结果与输出正确。

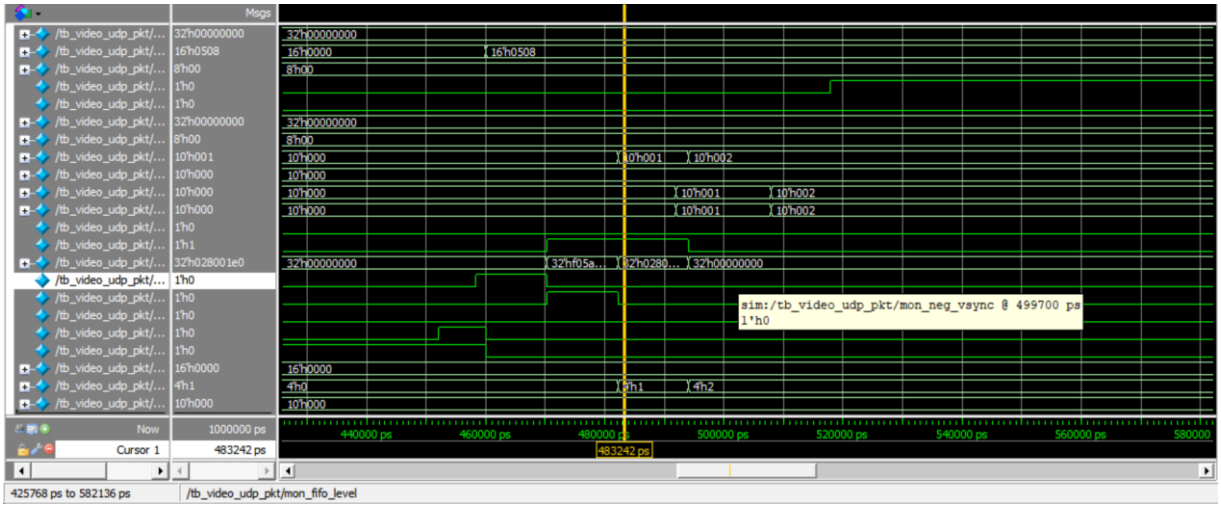


图 30 720p 分辨率以太网传输仿真

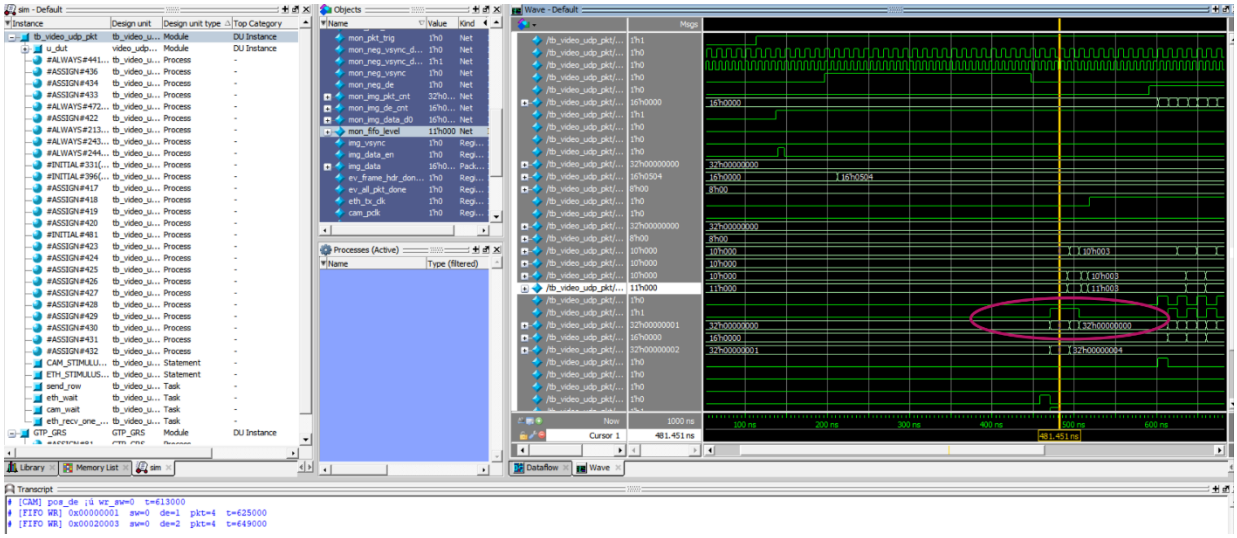


图 31 640×480 以太网传输仿真

车牌框选定位模块像素遍历与寄存器功能流程验证

1.模块功能概述

super_plate_tracker 是**车牌硬件追踪模块**，支持单帧图像内同时检测、追踪多组车牌目标。模块基于形态学二值化图像数据，通过像素遍历统计、动态势力范围匹配、长宽比约束、像素密度校验、四角极值算法，精准提取普通蓝牌与长条绿牌与黄牌的四角坐标，同时过滤环境噪点、碎片化干扰，最终输出有效车牌的四角坐标与有效标志位。在识别过程中支持 40 像素断裂缝隙容错、噪点密度过滤、对绿牌的尺寸自适应矫正、帧末统一输出结果，完全适配倾斜、断裂、长短不一的各类车牌场景。

2.仿真验证逻辑

仿真双目标车牌追踪逻辑，验证核心逻辑如下：

- 验证像素扫描逻辑：模块是否可逐像素接收二值化有效白点，完成目标轮廓实时统计。
- 验证双目标分离追踪逻辑：可同时区分、追踪两个无重叠车牌目标，无坐标串扰。
- 验证缝隙容错机制：支持车牌边缘 40 像素内断裂、空缺，不丢失目标。
- 验证双重过滤机制：通过**长宽比约束+像素密度校验**，过滤零散噪点、小面积干扰块。
- 验证四角极值提取算法：通过 sum、diff 差值算法精准提取车牌左上、右上、左下、右下四个角点坐标。
- 验证帧末刷新机制：单帧图像扫描完成后统一输出结果，同步清空寄存器，保证下一帧检测准确性。

3.仿真结果分析

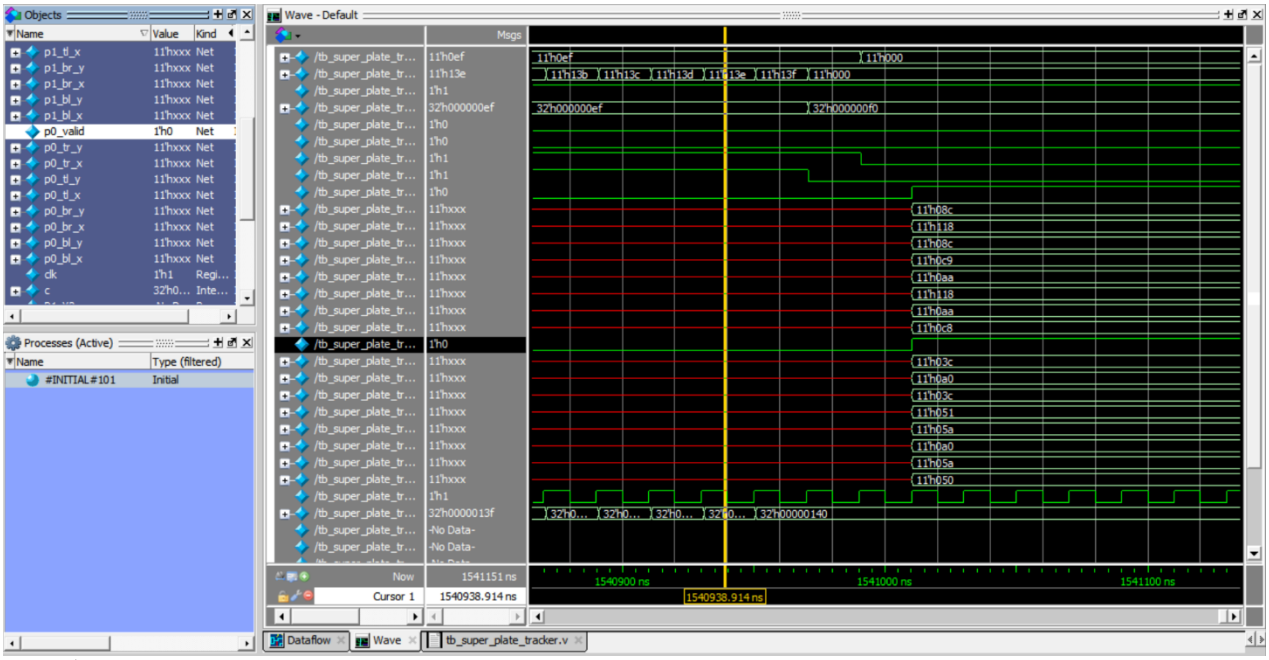
仿真过程与预期结果：

- 在经过电平初始化后，测试平台调用 send_frame 任务，向 DUT 注入一帧完整的视频数据。
- 帧头处将 vsync 拉高并保持，通知 DUT 新一帧开始，DUT 在此时清零内部的边界累积寄存器。而后按照从上到下从左到右进行像素点发送，每个像素携带坐标 x_cnt/y_cnt 和二值数据 per_img_Bit。在预设的两个矩形区域内 bit=1，其余位置 bit=0。每行结束后插入一拍消隐。
- 在帧尾将 vsync 拉低产生下降沿，DUT 检测到帧结束信号后将本帧累积的边界结果锁存到输出寄存器。

- 在像素扫描期间，DUT 对每个 clken=1 且 bit=1 的像素持续执行边界更新，用当前坐标不断刷新各候选框的最小 x、最大 x、最小 y、最大 y。帧结束时将这四个边界值组合成四个角点坐标并置 valid=1 输出。
- 帧结束后测试平台额外等待 8 拍（3 拍锁存等待加 5 拍稳定确认），然后读取输出端口并打印。预期结果为 p0 的四个角点对应 x:80-160、y:60-90 的矩形，p1 对应 x:200-280、y:140-170 的矩形，两个 valid 信号均为 1。

4.仿真输出：

读波形图及 tcl 输出发现像素点遍历范围 x（0-320），y（0-240），找到 P0：x 从 80 到 160，y 从 60 到 90，模拟第一块车牌；P1：x 从 200 到 280，y 从 140 到 170，模拟第二块车牌。预期与仿真结果对应，仿真结果正确。




```
# ========================================
# p0_valid=1  p1_valid=1
# PLATE0 TL(81,60) TR(160,60) BL(80,90) BR(160,90)
# PLATE1 TL(201,140) TR(280,140) BL(200,170) BR(280,170)
# ========================================
```

图 32 车牌识别仿真

5. 总结

5.1 可扩展之处

1. 在车牌颜色的识别方面，采用 YCbCr 和 HSV 做协同后进行判定，适用范围有限受噪声及光线影响较大，需要对光线自适应采用相关判别和补偿算法，提高系统识别稳定性准确率，尝试采用包括：同态滤波、二维伽马校正等通过查找表和流水线结构完成扩展。

2. 以太网发送接收模块我们现在只配置了 640×480 分辨率下的图像发送。因为分包简单，不存在行分包问题所以实现较为简单，我们通过运行仿真做实际 FIFO 复位及行结束包结束信号的观察逐步实现 1280×720 分辨率下的以太网打包发送。

3. 在 FPGA 单侧端我们也尝试采用字符分割，模对应等用于识别黑色白色的特殊车牌，但因背景颜色干扰较多，较难实现准确框选所以没有完成效果稳定准确的版本，需要做调研尝试部署 FPGA 端的包括神经网络在内的多种算法攻克。

5.2 心得体会

通过这个项目的开发，我们深刻体会到 FPGA+ARM 项目的开发不仅仅是编写 Verilog 代码，进行模型部署，更是一种系统性思维的锻炼。通过仿真验证，debug，逐帧观察波形定位具体问题等，我们完善了项目开发过程中的调试流程。通过不同方法的测试与设计，我们意识到在 FPGA 开发中片上资源的重要性，我们尝试部署轻量级神经网络，尝试用传统连通域分析并查集等划分区域的方法识别更多车牌发现实时性较差，片上资源严重消耗。通过 FPGA+ARM+GPU 的异构计算开发使用我们也完整了解了相关全流程，明白实时性和智能化的平衡是关键所在。

6. 主要代码解析

本系统包含“车牌识别”与“行人闯红灯预警”两个核心硬件工程。为了提高 FPGA 底层代码的复用率与系统稳定性，两个工程在底层架构上共享了多套图像处理与缓冲模块。本节将首先介绍两大工程共用的基础支撑模块，随后对各自特有的业务逻辑模块进行详细剖析。

1. 全局像素坐标系生成模块

共有模块 1：原始图像域坐标计数器

```
reg [10:0] raw_x, raw_y;
wire raw_vs_neg = ~raw_vs_r    & raw_vs_r2;   // 检测场同步下降沿
wire raw_de_neg = ~de_in_test & raw_de_r;   // 检测行数据有效下降沿

always @(posedge pclk_in_test or negedge rstn_out) begin
    if(!rstn_out)            begin raw_x<=0; raw_y<=0; end
    else if(raw_vs_neg) begin raw_x<=0; raw_y<=0; end              // 新的一帧，坐标清零
    else if(raw_de_neg) begin raw_x<=0; raw_y<=raw_y+1'b1; end// 换行，X 清零，Y 累加
    else if(de_in_test) begin raw_x<=raw_x+1'b1; end               // 行内数据有效，X 累加
end
```

2. 图像二值化与形态学滤波

共有模块 2：二值化与形态学级联流水线

```
// 1. 二值化处理：将提取出的车牌灰度掩码转化为 1-bit 黑白特征
wire bin_vsync, bin_hsync, bin_de, bin_bit;
binarization u_binarization(
    /* 端口连线省略 */ .per_img_Y(final_filtered_y), .Binary_Threshold(8'd130)
);

// 2. 腐蚀算子：收缩高亮区域，吃掉画面中的孤立小噪点
wire ero_vsync, ero_hsync, ero_de, ero_bit;
VIP_Bit_Erosion_Detector u_Erosion(
    /* 端口连线省略 */ .per_img_Bit(med_bit), .post_img_Bit(ero_bit)
);

// 3. 膨胀算子：连续调用 3 次膨胀模块，使断裂的车牌字符重新连通为一个整体
VIP_Bit_Dilation_Detector u_Dilate_1( /* 输入 ero_bit，输出 d1_bit */ );
VIP_Bit_Dilation_Detector u_Dilate_2( /* 输入 d1_bit，输出 d2_bit */ );
VIP_Bit_Dilation_Detector u_Dilate_3( /* 输入 d2_bit，输出 final_bit */ );
```

3. 一阶 IIR 坐标平滑数字滤波

共有模块 3：IIR 坐标平滑防抖与生命周期状态机

```
always @(posedge pclk_in_test) begin
    if(frame_end && p0_valid) begin
        if(!p0_init_done) begin
            // 首帧初始化：左移 2 位，构建定点数格式，防止小数精度丢失
            smooth_p0_tl_x <= {3'b0, p0_tl_x} << 2;
        end
        else begin
            // IIR 核心运算：利用移位>>2 衰减历史值，加上新坐标。约 4 帧收敛。
            smooth_p0_tl_x <= smooth_p0_tl_x - (smooth_p0_tl_x>>2) + {3'b0, p0_tl_x};
        end
        // 稳定确认：连续 3 帧有效才判定为目标真正出现（p0_stable）
        if(p0_stable_cnt < 4'd15) p0_stable_cnt <= p0_stable_cnt + 4'd1;
        p0_stable <= (p0_stable_cnt >= 4'd3);
    end
end
// 定点数还原：右移 2 位去除小数位，还原为物理屏幕坐标
wire [10:0] s_p0_tl_x = smooth_p0_tl_x[12:2];
```

4. DDR3 跨时钟域显存缓冲架构

共有模块 4：DDR3 跨时钟域显存缓冲架构

```
fram_buf fram_buf(
    // 【时钟域 A】：DDR3 核心物理控制时钟域
    .ddr_clk(core_clk),
    .ddr_rstn(ddr_init_done),

    // 【时钟域 B】：摄像头视频采集输入时钟域 (Write Channel)
    .vin_clk(pclk_in_test),
    .wr_fsync(~vs_in_test), // 写地址复位：利用摄像头 VSYNC 保证每帧首像素严格对齐
    .wr_en(de_in_test),         // 写数据使能
    .wr_data(final_rgb),        // 写入数据：已叠加好红色违章边框的最终图像

    // 【时钟域 C】：HDMI 高清视频显示输出时钟域 (Read Channel)
    .vout_clk(pix_clk),
    .rd_fsync(vs_reg),          // 读地址复位：受控于 HDMI 驱动生成的 VSYNC
    .rd_en(rd_en),              // 读数据使能
    .vout_de(de_o),
    .vout_data(o_rgb565),       // 读出数据：送往 HDMI 接口渲染的像素流

    // 【AXI4 内存映射主设备接口】(Master Interface)
    // 负责与底层硬核 DDR 控制器进行高吞吐量的 Burst 猝发握手
    .axi_awaddr(axi_awaddr), .axi_awvalid(axi_awvalid), .axi_awready(axi_awready),
```

```
    .axi_wdata(axi_wdata),     .axi_wlast(axi_wusero_last), .axi_wready(axi_wready),
    .axi_araddr(axi_araddr), .axi_arvalid(axi_arvalid), .axi_arready(axi_arready),
    .axi_rdata(axi_rdata),     .axi_rlast(axi_rlast),         .axi_rvalid(axi_rvalid)
    /* 省略部分辅助连线... */
);
```

5. 红绿灯色彩感知

特有模块 1：红绿灯感知状态机

```
wire in_tl_roi = (raw_x > 800) && (raw_y < 200);   // 圈定右上角感兴趣区域(ROI)
// 极红特征匹配：色相在首尾两端，饱和度与明度必须大于 100
wire is_red_pixel = (img_h > 9'd340 || img_h < 9'd20) && (img_s > 8'd100) && (img_v > 8'd100);

always @(posedge clk) begin
    if (vs_neg) begin
        // 帧结束时结算：单帧内红色像素大于 100 个，判定为红灯禁行状态
        if (red_cnt > 100) is_red_state <= 1'b1;
        red_cnt <= 0; // 计数器清零，准备下一帧
    end else if (de_in && in_tl_roi && is_red_pixel) begin
        red_cnt <= red_cnt + 1; // 在有效数据区且匹配颜色时，像素计数器累加
    end
end
```

6. BRAM 动态帧差追踪与防抖机制

特有模块 2：BRAM 帧差计算与全局切镜/抑制手抖

```
// 计算当前帧与 BRAM 中历史帧的绝对亮度差
wire [3:0] abs_diff = (curr_y_d1 > prev_y) ? (curr_y_d1 - prev_y) : (prev_y - curr_y_d1);
// 门控触发：亮度差 > 4 (滤除光扰底噪) 且处于红灯状态下，才输出原始运动像素
wire motion_raw = (abs_diff > 4'd4) & de_d1 & is_red_state;

always @(posedge clk) begin
    if (vs_neg) begin
        // 切镜免疫机制：如果一帧内发生运动的像素总数大于 20000 个
        // 判定为摄像头被人为晃动或环境剧变，拉高异常标志位，本帧提取的坐标直接作废
        is_camera_shake <= (global_motion_cnt > 20000) ? 1'b1 : 1'b0;
        global_motion_cnt <= 0;
    end else if (motion_raw) begin
        global_motion_cnt <= global_motion_cnt + 1; // 统计全局突变像素
    end
end
```

7. 车牌特化特征捕捉与边界检测

特有模块 3：车牌特化形态学与竖直边缘检测

```
// 1. 特化水平膨胀：利用 RADIUS=12 的超大搜索半径，专攻车牌水平缝隙，
//      强行将独立的中英文字符在水平方向上“糊”成一个完整的连续矩形。
wire hdil_vs, hdil_hs, hdil_de, hdil_bit;
VIP_Bit_HDilation #(.RADIUS(12)) u_HDilation(
    /* 接收最终膨胀数据 */ .per_img_Bit(final_bit),
    /* 输出水平连通数据 */ .post_img_Bit(hdil_bit)
);

// 2. 竖直 Sobel 边缘提取：仅计算水平方向的像素梯度(提取垂直边缘)。
//      阈值设为 3，削弱了车身横向装饰条的干扰，精准定位车牌边界。
wire sobel_vs, sobel_hs, sobel_de, sobel_bit;
VIP_Vertical_Sobel #(.THRESHOLD(3'd3)) u_VSobel(
    /* 接收水平连通数据 */ .per_img_Bit(hdil_bit),
    /* 输出高频垂直边缘 */ .post_img_Bit(sobel_bit)
);
```

7. 参考文献

[1]深圳市紫光同创电子有限公司. Logos2 系列 FPGA 器件数据手册：DS04001 V1.5[S]. 2020.

[2]深圳市小眼睛科技有限公司. MES2L676-200HP 硬件使用手册[M]. v1.2. 深圳: 深圳市小眼睛科技有限公司, 2024.

[3] 深圳市小眼睛科技有限公司. MES-FMC-LPC-IO 子卡硬件使用手册[M]. v1.1. 深圳: 深圳市小眼睛科技有限公司, 2025.

[4] OmniVision Technologies Inc. *OV5640 datasheet: 1/4-Inch color CMOS QSXGA (5 Megapixel) image sensor with OmniBSI™ technology* (version 2.01) [EB/OL]. (2011-01) [2024-11-20]. https://www.ovt.com/sensors/OV5640.

[5] S. Chen, Q. Song, H. Guo, and X. Li, "Design of license plate recognition system based on FPGA," in *2022 2nd Asia-Pacific Conference on Communications Technology and Computer Science (ACCTCS)*, Shenyang, China, 2022, pp. 227-231. doi: 10.1109/ACCTCS53867.2022.00054.

[6]正点原子仓库地址：https://github.com/alientek-openedv/Products.git

8. AI 使用说明

本团队在系统的研发与文档撰写过程中，严格遵守学术诚信与比赛规范。为了提高研发效率和突破技术瓶颈，我们在部分环节合理地借助了 AI 工具的辅助。特此声明本作品中 AI 工具的具体使用情况如下：

1. 创意与方案辅助

使用的 AI 工具：ChatGPT 5.0 / Gemini 3.1pro / Claude 4.6

1) 特定颜色阈值难以精准界定：在进行车牌颜色检测的初期，我们难以仅凭理论计算来确定目标颜色在 YCbCr/HSV 色彩空间中的精准阈值范围。

AI 启发我们：AI 辅助我们编写了一套轻量级的 Python 颜色调节器 GUI 脚本。通过该可视化工具，我们可以直观地滑动拖拽参数，实时观测不同阈值组合下对实时画面的色彩过滤效果。这加速了我们在 FPGA 硬件端色彩状态机的阈值标定工作，避免盲目试错。

2) 帧差法历史帧缓存的架构选型：在实现纯硬件底层帧差计算时，我们需要缓存上一帧的亮度数据。团队最初设想使用 FIFO 来作为缓存队列。

AI 启发我们：AI 在分析时序风险后指出，标准 FIFO 在一个像素的读写错位，整个画面的帧差对齐就会彻底崩溃。因此 AI 建议我们采用带有读写地址指针的 BRAM 来替代 FIFO，这样能保证前后两帧数据在物理空间坐标上的绝对严格对齐，大幅提升了视频流处理的容错率。

2. 文档撰写与语言润色

团队成员在撰写文档初稿时，逻辑较为口语化。我们将段落交由 AI 润色，AI 帮助我们实现了语言规范严谨化。

3. AI 生成内容占比声明：

文字内容：技术文档正文 AI 润色占比约 10%，核心技术描述、结论与分析均为团队原创撰写。

代码内容：AI 仅用于辅助生成伪代码，python 模拟分析，提供时序优化建议，核心算法、状态机与顶层控制逻辑均为团队独立实现，AI 直接生成的代码占比不足 5%。

设计与数据：系统架构、硬件设计方案、测试数据及实验图表均为团队独立设计与验证，无 AI 直接生成内容。

声明：AI 仅作为辅助工具使用，报告中的所有技术架构、代码实现、测试数据及图表，均由团队成员原创产出。